CS606-Compiler Construction

MID TERM MCQS

Prepared by: JUNAID MALIK



AL-JUNAID TECH INSTITUTE

Question No 1

LL(1) parsing is called _____ parsing.

- **Predictive**
- Non-predictive
- Post-predictive
- Post determine

Question No 2

One kind of predictive parser is the _____.

- **Recursive descent parses**
- Bottom-up parser
- Natural parser
- Multilingual parser

Question No 3

Front-end of a two pass compiler consists of _____.

- Code optimizer
- **Scanner**
- Parser
- Assembler

Question No 4

The algorithm for _____ conversion is called Thompson's Construction.

- DFA to RE
- **RE to DFA**
- RE to NFA
- CFG to NFA

Question No 5

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

_____ convert the relocatable machine into absolute machine code by linking library and relocatable object files.

- Assembler
- **Loader/link editor**
- Compiler
- Preprocessor

Question No 6

The _____ parser uses a stack to hold the frontier.

- Top-down
- **Bottom-up**
- Predicative
- LL(1) parsers

Question No 7

_____ is the process of discovering a derivation for some sentence of a language.

- Compiling
- Scanning
- **Parsing**
- Interpretation

Question No 8

A _____ accepts a string if we can follow transitions labeled with characters in the string from start state to some accepting state.

- **Finite automata……(For More Visit)**
- Regular expression
- Regular language
- Acceptor

0304-1659294
www.vulmshelp.com

Question No 9

The general idea behind NFA-to-DFA construction is that each _____ state corresponds to a set of _____ states.

- **DFA, NFA**
- NFA, DFA
- NFA, FA
- CFG, NFA

Question No 10

Optimal registers allocation is _____ problem.

- NP-hard
- **NP-complete**
- Polynomial complete
- Multinomial complete

Question No 11

Majority of the algorithms employed in the back-end are _____.

- Polynomial time
- NP-Complete
- **Np**
- None of the given

Question No 12

In predictive parsing table, the rows are _____.

- **Non-terminals**
- Terminals
- Both non-terminals and terminals
- Neither non-terminals nor terminals

0304-1659294
www.vulmshelp.com

Question No 13

Predictive parser accepts _____ grammars.

- LL(n+)
- LL(j-)
- **LL(k)**
- LL(K++)

Question No 14

_____ and _____ actions are used by bottom-up parsing.

- Break and reduce
- Shift and break
- **Shift and reduce**
- Shift and expand

Question No 15

If the right-hand side of a production has k symbols, it has _____ placeholder positions.

- k
- **k+1**
- k+2
- k+3

Question No 16

Lexical analyzer function is not _____.

- Removing white space
- Removing constants, identifies and keywords
- **Removing comments**
- Register allocation

Question No 17

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

Reduce action _____ zero or more symbols from the stack.

- Pushes
- Pops
- **Both pushes and pops**
- Adds

Question No 18

Parser determines if the input is _____ well formed.

- Programmatically
- Logically
- **Syntactically**
- Mechanically

Question No 19

Grammar with the _____ property is called predictive grammar.

- LL(3)
- LL(2)
- **LL(1)**
- LL(n+)

Question No 20

The executable code typically runs as a _____ in an Operating System Environment.

- **Process**
- Threads
- Routine
- Function

Question No 21

In a transition table cells of the table contain the _____ state.

0304-1659294
www.vulmshelp.com

- Reject State
- **Next State**
- Previous State
- Same State

Question No 22

In parser the two LL stand(s) for _____.

- Left-to-right scan of input
- Left-most derivation
- **Left-to-right scan of input and left-most derivation**
- Right to left scan of input

Question No 23

The _____ builds intermediate Representation (IR) for source program.

- Scanner
- **Parser**
- Compiler
- Loader

Question No 24

Left factoring of a grammer is done to save the parser from _____.

- Location tracking
- **Front tracking**
- Back tracking
- Random tracking

Question No 25

In Flex specification file different sections are separated by _____.

0304-1659294 www.vulmshelp.com

- **%%**
- &&
- ##
- \\

Question No 26

Left factoring of a grammar is done to save the parser from _____.

- Location tracking
- Front tracking
- **Back tracking**
- Random tracking

Question No 27

Lexer and scanner are _____ phase of compiler.

- **Same**
- Different
- Backend
- Middle end

Question No 28

Back End of two-pass compiler uses _____ algorithm.

- O(n)
- (n log n)
- **NP Complete**
- O(n2)

Question No 29

The _____ contains a lot of unneeded information.

- **Parse tree**

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- Abstract syntax tree
- Complete tree
- Abstract syntax tree

Question No 30

A _____ start at the leaves and grows toward root of the parse tree.

- Top-down parser
- **Bottom-up parser**
- Vertical parser
- Horizontal parser

Question No 31

Typical compilation means programs written in high-level languages to low-level _____.

- **Object code PG # 06**
- Byted code
- Unicode
- Object Code and byte code

Question No 32

In compilation process, Hierarchical analysis is also called _____.

- Parsing
- **Syntax**
- Parsing and Syntax analysis o
- None of given

Question No 33

0304-1659294
www.vulmshelp.com

IR (Intermediate Representation) stores the value of its operand in _____.

- **Registers PG # 10**
- Memory
- Hard disk
- None Of given

Question No 34

A lexeme is a sequence of characters in the source program that is matched by the pattern for a _____.

- Linker
- **Token**
- Control flow
- None of given

Question No 35

Parsers take _____ as Input from lexical analyzer.

- Linker
- **Token**
- Instructions
- None of the given

Question No 36

What kind of abstract machine can recognize strings in a regular set?

- **DFA**
- NFA
- PDA
- None of the given

Question No 37

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

In multi pass compiler, during the first pass it gathers information about _____.

- **Declaration**
- Bindings
- Static information
- None of the given

Question No 38

In DFA minimization, we construct one _____ for each group of states from the initial DFA.

- **State PG # 30**
- NFA
- PDA
- None of given

Question No 39

_____(Lexical Analyzer generator), is written in Java..

- Flex
- **Jlex PG # 31**
- Complex
- None of given

Question No 40

Recursive _____ parsing is done for LL(I) grammar.

- **Decent**
- Ascent
- Forward
- None of the given

Question No 41

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

Alternative of the backtrack in parser is Look ahead symbol in _____.

- **Input**
- Output
- Input and Output
- None of the given

Question No 42

Parser takes tokens from scanner and tries to generate _____.

- Binary Search tree
- **Parse tree**
- Binary Search tree and Parse tree
- None of the given

Question No 43

In predictive parsing table, the rows represents _____.

- Terminals
- Both non-terminal and terminals
- **Non-terminals PG # 62**
- None of the given

Question No 44

In LL(I) parsing algorithm,_____ contains a sequence of grammar symbols.

- **Stack PG # 62**
- Link List
- Array
- None of the given

Question No 45

Consider the grammar

0304-1659294 www.vulmshelp.com

A ⟹ B C D

B ⟹ h B | £

C ⟹ C g | g | C h | i

D ⟹ AB | £

First of A is _____.

- **h, g, i**
- g
- h
- None of the given

Question No 46

Consider the grammar

A ⟹ B C D

B ⟹ h B | £

C ⟹ C g | g | C h | i

D ⟹ AB | £

First of C is _____.

- **g, i**
- g
- h, i
- None of the given

Question No 47

Bottom-up parsing uses only _____ kinds of actions.

- **Two PG # 71**
- Three
- Four

0304-1659294
www.vulmshelp.com

- Five

Question No 48

The shift action _____ a terminal on the stack.

- **Pushes PG # 73**
- Pops
- Both push and pops
- None of the given

Question No 49

Reduce _____ action zero or more symbols from the stack.

- Pushes
- **Pops PG # 73**
- Both push and pops
- None of the given

Question No 50

In compilers, linear analysis is also called _____.

- Lexical analysis
- Scanning
- **Lexical analysis and scanning**
- None of the given

Question No 51

Back End of two-pass compiler uses _____ algorithm.

- O(n)
- O(n log n)
- **NP Complete**
- None of the Given

Question No 52

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

The Back End of a compiler consist of _____.

- **Instruction selection**
- Register allocation
- Instruction scheduling
- All of the given

Question No 53

In Back End module of compiler, optimal register allocation uses _____.

- O(log n)
- O(n log n)
- **NP-Complete**
- None of the given

Question No 54

._____is a regular expression for the set of all strings over the alphabet {a} that has an even number of a' s.

- **(aa)***
- (aa)*a
- aa*a
- a*(aa)

Question No 55

_____algorithm is used in DFA minimization.

- James's
- Robert's
- **Hopcroft's PG # 25**
- None of given

Question No 56

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

_____ is an important component of semantic analysis.

- Code checking
- **Type checking PG # 39**
- Flush checking
- None of the given

Question No 57

Consider the grammar

A ⟶ B C D

B ⟶ h B | £

C ⟶ C g | g | C h | i

First of B is_____.

- h, i
- **h, £**
- g
- None of the given

Question No 58

Consider the grammar,

A ⟶ B C D

B ⟶ h B | £

C ⟶ C g | g | C h | i

D ⟶ AB | £

First of D is_____.

- h, g
- h
- **h, g , i, £**

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- None of the given

Question No 59

_____ Parsers never shifts into an error state.

- LS
- LT
- **LR**
- LP

Question No 60

In_____ , certain checks are performed to ensure that components of a program fit together meaningfully..

- Linear analysis
- Hierarchical analysis
- **Semantic analysis**
- None of given

Question No 61

_____ read the input character and produce sequence of tokens as output.

- **Lexical analyzer**
- Parser
- Symbol table
- None of the given

Question No 62

. The regular expression _____denotes, the set of all strings of a's and b's of length two.

- a*
- (a* | b*)*

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- (a*b*)*
- **(a|b)(a|b)**

Question No 63

_____ of a two-pass compiler is consists of Instruction selection, Register allocation and Instruction scheduling.

- **Backend PG # 9**
- Frontend
- Start
- None of the Given

Question No 64

_____ is evaluated to yield a value.

- Command
- **Expression**
- Declaration
- None of the given

Question No 65

Bottom-up parsers handle a _____ class of grammar.

- **Large PG # 49**
- Small
- Medium
- None of the given

Question No 66

The input specification file to flax consists of-------- section.

- 1
- 2

0304-1659294 www.vulmshelp.com

- **3**
- 4

Question No 67

Lexer and scanner are two different phases of compiler.

- True
- **False**

Question No 68

In compiler linear analysis is also called.

- Lexical analysis
- Scanning
- **Both lexical analysis and scanning**
- None of the given

Question No 69

Which of the following statements is NOT true?

- The language accepted by finite automate are the languages denoted by regular expression.
- For every DFA there is a regular expression denoting its languages.
- **For a regular expression r, there does not exit NFA with L(r) any transit that accept.**
- None of the given.

Question No 70

Top – down parsing expands a ---------- from the start symbol to the leaves.

- Parse tree
- Abstract Syntax tree
- Parse tree
- **All of the given**

Question No 71

A top-down purser starts with the ---------- of the parse tree.

- **Root**
- Leaf
- Middle
- None of the given

Question No 72

Can a DFA simulate NFA?

- NO
- **YES**
- Sometimes
- Depends on NFA

Question No 73

Bottom- up parsing is also called----------.

- **LR parsing**
- LT parsing
- LS parsing
- SS parsing

Question No 74

The ---------- is optimized for hardware it is to run on.

- C++ code
- C code
- **Assembly code**
- None of the given

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

Question No 75

Abstract syntax tree summarizes ---------- without the details of derivation.

- Theory
- **Grammatical structure**
- Data
- None of the give

Question No 76

LR parsing---------- a string to the start symbol by inverting production.

- **Reduces**
- Shifts
- Adds
- None of the given

Question No 77

Alternative to backtrack in top- down parser is ----------.

- Context free grammar
- Trees
- **Look ahead**
- None of the given

Question No 78

Typical compilation means programs written in high- level languages to low-languages----------.

- **Object code**
- Byte Code
- Unicode
- Both object code and Byte code

Question No 79

0304-1659294
www.vulmshelp.com

The ---------- Checks the stream of work (token) and their parts of speech for grammatical correctness.

- Parser
- **Scanner**
- **Compiler**
- None of the given

Question No 80

A grammar must be ---------- before use for predicative parsing.

- Right factored
- **Left factored**
- Factored
- None of the given

Question No 80

Parser does not distinguish between valid and invalid sequences of token.

- True
- **False**

Question No 81

In Compilation process Hierarchical analysis is also called:

- Parsing
- **Syntax analysis**
- Both parsing and Syntax analysis
- None of the given

Question No 82

The pair <role, word> is given the name----------.

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- Word
- **Token**
- Syntax
- None of the given

Question No 83

The back end maps intermediate Representation (IR) into target----------.

- Object code
- Machine code
- **Source code**
- Linker

Question No 84

---------- is evaluated to yield a value.

- Command
- **Expression**
- Declaration
- Declaration and Command

Question No 85

Front- end of a two pass compiler is consists of scanner.

- **TRUE**
- FALSE

Question No 86

The ---------- returns a sequence of mating token at output (or an error) and it always return the longest matching token.

- Scanner
- Lexer
- **Lexical analyzer**

0304-1659294
www.vulmshelp.com

- All of the given

Question No 87

---------- avoid hardware stalls and interlocks.

- Register allocation
- **Instruction scheduling**
- Instruction selection
- None of given

Question No 88

LR parsers can handle _____ grammars.

- Left-recursive (Page 63)
- file-recursive
- End-recursive
- Start-recursive

Question No 89

_____ convert the relocatable machine code into absolute machine code by linking library and relocatable object files.

- Assembler
- Loader/link-editor
- Compiler
- Preprocessor

Question No 90

Follow of C is _____ .

- g, h, i, $
- g, h, $
- h, i, $

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- h, g, $

Question No 91

Consider the grammar A --> B C D

B --> h B | epsilon

C --> C g | g | C h | i

D --> A B | epsilon

Follow of B is _____.

- **h**
- g, h, i, $
- g, i
- g

Question No 92

An important component of semantic analysis is _____.

- Code checking
- **type checking (Page 33)**
- flush checking
- None of the given

Question No 93

In PASCAL _____ represent the inequality test.

- :=
- =
- <>
- None of the given

0304-1659294
www.vulmshelp.com

Question No 94

Lexical Analyzer generator _____ is written in Java.

- Flex
- **Jlex (Page 26)**
- Complex
- None of given

Question No 95

_____avoid hardware stalls and interlocks.

- Register allocation
- **Instruction scheduling (Page 121)**
- Instruction selection
- None of given

Question No 96

Consider the following grammar,

A --> B C D

B --> h B | epsilon

C --> C g | g | C h | i

D --> A B | epsilon

First of A is _____.

- h, g, i
- g
- **h**
- None of the given

0304-1659294
www.vulmshelp.com

Question No 97

Recursive _____ parsing is done for LL(1) grammar.

- **Descent (Page 47)**
- Ascent
- Forward
- Backward

Question No 98

One of the core tasks of compiler is to generate fast and compact executable code.

- **True**
- False

Question No 99

Left factoring of a grammar is done to save the parser from back tracking.

- **True**
- False

Question No 100

Responsibility of _____ is to produce fast and compact code.

- **Instruction selection (Page 9)**
- Register allocation
- Instruction scheduling
- None of given

Question No 101

_____ algorithm is used in DFA minimization.

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- James's
- Robert's
- **Hopcroft's (Page 25)**
- None of given

Question No 102

Compilers are sometimes classified as.

- Single pass
- Multi pass
- Load and go
- **All of the given**

Question No 103

In multi pass compiler during the first pass it gathers information about _____.

- **Declaration**
- Bindings
- Static information
- None of the given

Question No 104

Flex is an automated tool that is used to get the minimized DFA (scanner).

- True
- **False (Page 26)**

Question No 105

In compilation process Hierarchical analysis is also called

- Parsing

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- Syntax analysis
- **Both Parsing and Syntax analysis**
- None of given

Question No 106

For each language to make LL(1) grammar, we take two steps, 1st is removing left recurrence and 2nd is applying left factoring in sequence.

- **True**
- False

Question No 107

_____is evaluated to yield a value.

- Command
- **Expression**
- Declaration
- Declaration and Command

Question No 108

Optimal registers allocation is an NP-hard problem.

- True
- **False**

Question No 109

Parser takes tokens from scanner and tries to generate _____ .

- Binary Search tree
- **Parse tree**
- Syntax trace
- None of the given

Question No 110

0304-1659294
www.vulmshelp.com

Front end of two pass compiler takes_____ as input.

- **Source code (Page 5)**
- Intermediate Representation (IR)
- Machine Code
- None of the Given

Question No 111

In in we construct one _____ for each group of states from the initial DFA.

- **State (Page 25)**
- NFA
- PDA
- None of given

Question No 112

In Three-pass compiler _____ is used for code improvement or optimization.

- Front End
- **Middle End (Page 10)**
- Back End
- Both Front end and Back end

Question No 113

_____ of a two-pass compiler is consists of Instruction selection, Register allocation and Instruction scheduling.

- **Back end (Page 10)**
- Front end
- Start
- None of given

Question No 114

NFA is easy to implement as compared to DFA.

- True
- **False (Page 19)**

Question No 115

We can get an LL(1) grammar by _____ .

- Removing left recurrence
- Applying left factoring
- **Removing left recurrence and Applying left factoring**
- None of the given

Question No 116

Parser always gives a tree like structure as output

- True
- False

Question No 117

In Back End module of compiler, optimal register allocation uses_____ .

- O(log n)
- O(n log n)
- **N P-Complete (Page 10)**
- None of the given

Question No 118

Lexer and scanner are two different phases of compiler
Select correct option:

- True
- **False**

0304-1659294
www.vulmshelp.com

Question No 119

In Flex specification file different sections are separated by _____.

- **%% (Page 26)**
- &&
- ##
- \\

Question No 120

_____ phase which supports macro substitution and conditional compilation.

- Semantic
- Syntax
- **Preprocessing** Click here for detail
- None of give

Question No 121

_____tree in which each node represents an operator and children of the node represent the operands.

- **Abstract syntax (Page 100)**
- Concrete syntax
- Parse
- None of the given

Question No 122

In _____certain checks are performed to ensure that components of a program fit together meaningfully.

- Linear analysis
- Hierarchical analysis
- **Semantic analysis (Page 33)**
- None of given

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

Question No 123

In compiler linear analysis is also called.

Select correct option:

- Lexical analysis
- Scanning
- **Both lexical analysis and scanning click here for details**
- None of the given

Question No 124

Which of the following statement is true about Two pass compiler.

- Front End depends upon Back End
- **Back End depends upon Frond End (Page 5)**
- Both are independent of each other
- None of the given

Question No 125

Ambiguity can easily be handled by Top-down Parser

- **True click here for detail**
- False

Question No 126

Alternative of the backtrack in parser is Look ahead symbol in _____ .

- **Input (Page 46)**
- Output
- Input and Output
- None of the given

Question No 127

Typical compilation means programs written in high-level languages to low-level _____.

0304-1659294 www.vulmshelp.com

- Object code
- Byte code
- Unicode
- **Both Object Code and byte code (Page 2)**

Question No 128

LR parsing _____ a string to the start symbol by inverting.

- **Reduces (Page 63)**
- Shifts
- Adds
- None of the given

Question No 129

In parser the two LL stand(s) for _____.

- Left-to-right scan of input
- left-most derivation
- **All of the given click here for detail**
- **None of given**

Question No 130

Bottom-up parsers handle a _____ class of grammars.

- **Large (Page 63)**
- Small
- Medium
- None of the given

Question No 131

Compilers are some times classified as.

- Single pass
- Multi pass
- Load and go

0304-1659294
www.vulmshelp.com

- **All of the given**

Question No 132

The Back End of a compiler consist of _____ .

- **Instruction selection** (Page 10)
- Register allocation
- Instruction scheduling
- All of the given

Question No 133

Can a DFA simulate NFA?

- **Yes** click here for detail
- No
- Sometimes
- Depend upon nfa

Question No 134

Which of the statement is true about Regular Languages?

- Regular Languages are the most popular for specifying tokens.
- Regular Languages are based on simple and useful theory.
- Regular Languages are easy to understand.
- **All of the given (Page 15)**

Question no 135

In a transition table cells of the table contain the _____ state.

- Reject state
- **Next state (Page 18)**
- Previous state
- None of the given

Question No 136

The transition graph for an NFA that recognizes the language (a | b)*abb will have following set of states.

- {0}
- {0,1}
- {0,1,2}
- **{0,1,2,3} (page 23) Not sure**

Question No 137

Functions of Lexical analyzer are?

- Removing white space
- Removing constants, identifiers and keywords
- Removing comments
- **All of the given** click here for detail

Question No 138

_____ method is known as Subset Construction Method.

- **NFA TO DFA (Page 19)**
- DFA
- DFA maximization
- None of given

Question No 139

Which one is NOT the Role of Run-Time System

- garbage collection
- memory management
- run time error checking
- **none of given (Page 11)**

Question No 140

Compiler_____ information from one representation to another

- modified
- **translate (Page 2)**

- execute
- extract

Question No 141

Front-end of a two pass compiler is consists of Scanner.

- True
- **False (Page 12)**

Question No 142

Bottom-up parsing uses only _____ kinds of actions.

- **Two (Page 64)**
- Three
- Four
- Five

Question No 143

Bottom-up parsing is also called _____ .

- **LR parsing (Page 63)**
- LT parsing
- LS parsing
- SS parsing

Question No 144

Backtracking is a costly operation which is caused due to Left Recursion.

- True
- **False**

Question No 145

A non-recursive predictive parser is also called _____.

- **table-driven parser (Page 54)**

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

- Abstract parser
- Conceptual parser
- None of the given

Question No 146

AST summarizes the grammatical structure with the details of derivations.

- True
- **False (Page 8)**

Question No 147

Backtrack is term associated with Top-down parsing.

- **True (Page 42)**
- False

Which of the following statement is true about Two-pass compiler.

- Front End depends upon Back End
- Back End depends upon Front End
- Both are independent of each other
- None

2. In a three-pass compiler, ___ is used for code improvement or optimization.

- Front End
- Middle End
- Back End
- Both Front End and Back End

3. In multi-pass compiler during the first pass, it gathers information about ___

- Declaration
- Bindings
- Static information

- None

4. Which of the statement is true about Regular Languages?

- Regular Languages are the most popular for specifying tokens.
- Regular Languages are based on simple and useful theory.
- Regular Languages are easy to understand
- All of the given

5. The DFA uses its state to keep track of ___ the NFA can be in after reading each input symbol.

- Accept state
- Reject state
- Next state
- All possible states

6. In ___ certain check are performed to ensure that components of a program fit together meaningfully.

- Linear analysis
- Hierarchical analysis
- Semantic analysis
- None

7. Compilers are some times classifies as:

- Single-pass
- Multipass
- Load and go
- All of the given

8. In compiler, linear analysis is also called

- Lexical analysis
- Scanning
- Both lexical analysis and scanning
- None

9. In a transition table cells of the table contain the ___ state.

- Reject state
- Next state
- Previous state
- None

10. In Back End module of the compiler, optimal register allocation uses ___.

- O(log n)
- O(n log n)
- NP-Complete
- None

11. A non-recursive predictive parser is also called ___

- table-driven parser
- Abstract parser
- Conceptual parser
- None

12. The shift action ___ a terminal on the stack

- Pushes
- Pops
- Both pushes and pops
- None

13. Bottom-up parsing is also called ___

- LR parsing
- LT parsing
- LS parsing
- SS parsing

14. ___ of a two-pass compiler is consists of instruction selection, Register allocation and instrcution scheduling

- Backend
- Frontend
- Start

- None

15. Consider the grammar
A -> B C C
B -> h B | epsilon
C -> C g | g | C h | i
D -> A B | epsilon
First of A is ___

- h, g, i
- g
- h
- None

16. A -> B C C
B -> h B | epsilon
C -> C g | g | C h | i

D -> A B | epsilon
Follow of A ___

- h, $
- $
- i
- g

17. A -> B C C
B -> h B | epsilon
C -> C g | g | C h | i
D -> A B | epsilon
Follow of C is ___

- g, h, i, $
- g, h, $
- h, i, $
- h, g, $

18. Left factoring of grammar is done to save the parser from backtracking.

- True
- False

19. Typical compilation means programs written in high-level languages to low-level ___

- Object code
- Byte code
- Unicode
- Object code and byte code

20. In compilation process, Hierarchical analysis is also called ___

- Parsing
- Syntax
- Parsing and Syntax analysis
- None

21. IR (Intermediate Representation) stores the value of its operand in ___

- Registers
- Memory
- Hard disk
- None

22. A lexeme is a sequence of characters in the source program that is matched by the pattern for a ___

- Linker
- Token
- Control flow
- None

23. Parsers take ___ as Input from lexical analyzer.

- Linker
- Token
- Instructions
- None

24. What kind of abstract machine can recognize strings in a regular set?

- DFA
- NFA
- PDA
- None

25. In DFA minimization, we construct one ___ for each group of states from the initial DFA.
- State
- NFA
- PDA
- None

26. ___ (lexical Analyzer generator), is written in Java
- Flex
- Jlex
- Complex
- None

27. In flex specification file, different sections are separated by ___
- %%
- &&
- ##
- None

28. Recursive ___ parsing is done for LL(I) grammar.
- Descent y
- Ascent
- Forward
- None

29. Alternative of the backtrack in parser is Look ahead symbol in ___
- Input
- Output
- Input and Output
- None

30. Parser takes tokens from scanner and tries to generate ___
- Binary Search Tree
- Parse tree
- Binary Search and Parse tree
- None

31. In predictive parsing table, the rows represents ___
- Terminals
- Both non-terminals and terminals

- Non-terminals
- None

32. In LL(I) parsing algorithm, ___ contains a sequence of grammar symbols.
- Stack
- Link list
- Array
- None

33. Consider the grammar
A -> B C D
B -> h B | epsilon
C -> C g | g | C h | i
D -> A B | epsilon
First of C is ___
- g, i
- g
- h, i
- None

34. Bottom-up parsing uses only ___ kinds of actions.
- Two
- Three
- Four
- Five

35. Reduce ___ action zero or more symbols from the stack.
- Pushes
- Pops
- Both push and pops
- None

36. Back End of two-pass compiler uses ___ algorithm
- O(n)
- O(n log n)
- NP complete
- None

37. The Back End of a compiler consists of ___
- Instruction selection
- Register allocation

- Instruction scheduling
- All of given

38. ___ is a regular expression for the set of all strings over the alphabet {a} that has an even number of a's

- aa*
- (aa)*
- aa*a
- a(aa)*

39. ___ algorithm is used in DFA minimization.

- Jame's
- Robert's
- Hopcroft's y
- None

40. ___ is an important component of semantic analysis.

- Code checking
- Type checking
- Flush checking
- None

41. Consider the grammar
A – > B C D
B -> h B | epsilon
C -> C g | g | C h | i
First of B is ___

- h, i
- h, epsilon
- g
- None

42. Consider the grammar
A -> B C D
B -> h B | epsilon
C -> C g | g | C h | i
D -> A B | epsilon
First of D is ___

- h, g
- h

- h, g, i, epsilon
- None

43. ___ parsers never shifts into an error state.
- LS
- LT
- LR
- LP

44. ___ read the input character and produce sequence of tokens as output.
- Lexical analyzer
- Parser
- Symbol table
- None

45. The regular expression ___ denotes, the set of all strings of a's and b's of length two.
- a*
- (a*| b*)*
- (a*b*)*
- (a|b)(a|b)

46. ___ is evaluated to yield a value.
- Command
- Expression
- Declaration
- None

47. Bottom-up parsers handle a ___ class of grammar
- Large
- Small
- Medium
- None

48. LR parsers can handle ___ grammars.
- Left-recursive
- File-recursive
- End-recursive
- Start-recursive

49. ___ convert the relocatable machine code into absolute machine code by linking library and relocatable object files.

- Assembler
- Loader/link-editor
- Compiler
- Preprocessor

50. Consider the grammar
A -> B C D
B -> h B | epsilon
C -> C g | g | C h | i
D -> A B | epsilon
Follow of B is ___

- h
- g, h, i, $
- g, i
- g

51. In PASCAL ___represent the inequality test.

- :=
- =
- <>
- None

52. ___ avoid hardware stalls and interlocks.

- Register allocation
- Instruction scheduling
- Instruction selection
- None

53. One of the core tasks of compiler is to generate fast and compact executable code.

- True
- False

54. Responsibility of ___ is to produce fast and compact code.

- Instruction selection
- Register allocation
- Instruction scheduling
- None

55. Compilers are sometimes classified as

- Single pass

- Multi pass
- Load and go
- All of given

56. Flex is an automated tool that is used to get the minimized DFA (scanner).

- True
- False

57. For each language to make LL(1) grammar, we take two steps, 1st is removing left recurrence and 2nd is applying left factoring in sequence.

- True
- False

58. Optimal registers allocation is an NP-hard problem.

- True
- False

59. Front end of two pass compiler takes ___ as input.

- Source code
- Intermediate Representation (IR)
- Machine code
- None

60. NFA is easy to implement as compared to DFA

- True
- False

61. We can get an LL(1) grammar by ___

- Removing left recurrence
- Applying left factoring
- Removing left recurrence and Applying left factoring
- None

63. Lexer and scanner are two different phases of compiler.

- True
- False

64. ___ phase which supports macro substitution and conditional compilation.

- Semantic
- Syntax
- Preprocessing

- None

65. ___ tree in which each node represents an operator and children of the node represent the operands

- Abstract syntax
- Concrete syntax
- Parse
- None

66. Ambiguity can easily be handled by top-down parser

- True
- False

67. LR parsing ___ a string to the start symbol by inverting productions.

- Reduces
- Shifts
- Adds
- None

68. In parser the two LL stand(s) for ___

- Left-to-right scan of input
- left-most derivation
- All of the given
- None

69. Can a DFA simulate NFA?

- Yes
- No
- Sometimes
- Depend upon NFA

70. The transition graph for an NFA that recognized the language (a|b)*abb will have following set of states.

- {0}
- {0, 1}
- {0, 1, 2}
- {0, 1, 2, 3}

71. Functions of Lexical analyzer are?

- Removing white space
- Removing constants, identifiers and keywords

- Removing comments
- All of given

72. ___ method is known as subset construction method.
- NFA to DFA
- DFA
- DFA maximization
- None

73. Which one is NOT the Role of Run-Time System?
- garbage collection
- memory management
- run time error checking
- None

74. Compiler ___ information from one representation to another.
- modified
- translate
- execute
- extract

75. Front-end of a two pass compiler is consists of scanner.
- True
- False

76. Backtracking is a costly operation which is caused due to Left Recursion.
- True
- False

77. AST summarizes the grammatical structure with the details of derivations.
- True
- False

78. Backtrack is term associated with Top-down parsing.
- True
- False


AL-JUNAID TECH INSTITUTE
AJ
0304-1659294
www.vulmshelp.com

MIDTERM EXAMINATION
Fall 2012
CS606- compiler instruction

Question # 1 (Total Marks: 1) Select correct option:
LR parsers can handle _____ grammars.
Left-recursive (Page 63)
file-recursive
End-recursive
Start-recursive

Question # 2 (Total Marks: 1) Select correct option:
_____ convert the relocatable machine code into absolute machine code by linking library and relocatable object files.
Assembler
Loader/link-editor
Compiler
Preprocessor

Question # 3 (Total Marks: 1) Select correct option:
Follow of C is _____ .
g, h, i, $
g, h, $
h, i, $
h, g, $

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 4 (Total Marks: 1) Select correct option:
Consider the grammar A --> B C D
B --> h B | epsilon
C --> C g | g | C h | i
D --> A B | epsilon
Follow of B is _____ .

h
g, h, i, $
g, i
g

Question # 5 (Total Marks: 1) Select correct option:
An important component of semantic analysis is _____ .

code checking
type checking (Page 33)
flush checking
None of the given

Question # 6 (Total Marks: 1) Select correct option:
In PASCAL _____ represent the inequality test.

:=
=
<>
None of the given

Question # 7 (Total Marks: 1) Select correct option:
Lexical Analyzer generator _____ is written in Java.

Flex
Jlex (Page 26)
Complex
None of given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 8 (Total Marks: 1) Select correct option:
_____avoid hardware stalls and interlocks.

Register allocation
Instruction scheduling (Page 121)
Instruction selection
None of given

Question # 9 (Total Marks: 1) Select correct option:
Consider the following grammar,
A --> B C D
B --> h B | epsilon
C --> C g | g | C h | i
D --> A B | epsilon
First of A is _____ .
h, g, i
g
h
None of the given

Question # 10 (Total Marks: 1) Select correct option:
Recursive _____ parsing is done for LL(1) grammar.
Descent (Page 47)
Ascent
Forward
Backward

Question # 1 (Total Marks: 1) Select correct option:
One of the core tasks of compiler is to generate fast and compact executable code.
True
False

Question # 11 (Total Marks: 1) Select correct option:
Left factoring of a grammar is done to save the parser from back tracking.
True
False

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 12 (Total Marks: 1) Select correct option:
Responsibility of _____ is to produce fast and compact code.
Instruction selection (Page 9)
Register allocation
Instruction scheduling
None of given

Question # 13 (Total Marks: 1) Select correct option:
_____ algorithm is used in DFA minimization.
James's
Robert's
Hopcroft's (Page 25)
None of given

Question # 14 (Total Marks: 1) Select correct option:
Compilers are sometimes classified as.
Single pass
Multi pass
Load and go
All of the given click here for detail

Question # 15 (Total Marks: 1) Select correct option:
In multi pass compiler during the first pass it gathers information about _____ .
Declaration click here for detail
Bindings
Static information
None of the given

Question # 16 (Total Marks: 1) Select correct option:
Flex is an automated tool that is used to get the minimized DFA (scanner).
True
False (Page 26)

Question # 17 (Total Marks: 1) Select correct option:
In compilation process Hierarchical analysis is also called
Parsing
Syntax analysis
Both Parsing and Syntax analysis click here 4 detail
None of given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 18 (Total Marks: 1) Select correct option:
For each language to make LL(1) grammar, we take two steps, 1st is removing left recurrence and 2nd is applying left factoring in sequence.

True
False

Question # 19 (Total Marks: 1) Select correct option:
_____is evaluated to yield a value.
Command
Expression
Declaration
Declaration and Command

Question # 20 (Total Marks: 1) Select correct option:
Optimal registers allocation is an NP-hard problem.
True
False

MIDTERM EXAMINATION
Fall 2012
CS606- compiler instruction

Question # 1 (Total Marks: 1) Select correct option:
Parser takes tokens from scanner and tries to generate _____ .
Binary Search tree
Parse tree
Syntax trace
None of the given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 2 (Total Marks: 1) Select correct option:
Front end of two pass compiler takes_____ as input.
Source code (Page 5)
Intermediate Representation (IR)
Machine Code
None of the Given

Question # 3 (Total Marks: 1) Select correct option:
In in we construct one _____ for each group of states from the initial DFA.

State (Page 25)
NFA
PDA
None of given

Question # 4 (Total Marks: 1) Select correct option:
In Three-pass compiler _____ is used for code improvement or optimization.

Front End
Middle End (Page 10)
Back End
Both Front end and Back end

Question # 5 (Total Marks: 1) Select correct option:
_____ of a two-pass compiler is consists of Instruction selection, Register allocation and Instruction scheduling.

Back end (Page 10)
Front end
Start
None of given

Question # 6 (Total Marks: 1) Select correct option:
NFA is easy to implement as compared to DFA.
True
False (Page 19)

Question # 7 (Total Marks: 1) Select correct option:
We can get an LL(1) grammar by _____ .
Removing left recurrence
Applying left factoring
Removing left recurrence and Applying left factoring
None of the given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 8 (Total Marks: 1) Select correct option:
Parser always gives a tree like structure as output
True
False

Question # 9 (Total Marks: 1) Select correct option:
In Back End module of compiler, optimal register allocation uses_____ .

O(log n)
O(n log n)
N P-Complete (Page 10)
None of the given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

CS606 – Midterm Solved Quizzes (Quiz No.1 & 2)

Question # 1 of 10 (Total Marks: 1) Select correct option:
Lexer and scanner are two different phases of compiler
Select correct option:
True
False

Question # 2 of 10 (Total Marks: 1) Select correct option:
In Flex specification file different sections are separated by _____ .

%% (Page 26)
&&
##
\\

Question # 3 of 10 (Total Marks: 1) Select correct option:
_____ phase which supports macro substitution and conditional compilation.
Semantic
Syntax
Preprocessing Click here for detail
None of given

Question # 4 of 10 (Total Marks: 1) Select correct option:
_____tree in which each node represents an operator and children of the node represent the operands.
Abstract syntax (Page 100)
Concrete syntax
Parse
None of the given

Question # 5 of 10 (Total Marks: 1) Select correct option:
In _____certain checks are performed to ensure that components of a program fit together meaningfully.
Linear analysis
Hierarchical analysis
Semantic analysis (Page 33)
None of given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 6 of 10 (Total Marks: 1) Select correct option:
In compiler linear analysis is also called.
Select correct option:
Lexical analysis
Scanning
Both lexical analysis and scanning click here for details
None of the given

Question # 7 of 10 (Total Marks: 1) Select correct option:
Which of the following statement is true about Two pass compiler.
Front End depends upon Back End
Back End depends upon Frond End (Page 5)
Both are independent of each other
None of the given

Question # 8 of 10 (Total Marks: 1) Select correct option:
Ambiguity can easily be handled by Top-down Parser
True click here for detail
False

Question # 10 of 10 (Total Marks: 1) Select correct option:
Alternative of the backtrack in parser is Look ahead symbol in _____ .
Input (Page 46)
Output
Input and Output
None of the given

Question # 1 of 10 (Total Marks: 1) Select correct option:
Typical compilation means programs written in high-level languages to low-level _____.
Object code
Byted code
Unicode
Both Object Code and byte code (Page 2)

Question # 2 of 10 (Total Marks: 1) Select correct option:
LR parsing _____ a string to the start symbol by inverting productions.
Reduces (Page 63)
Shifts
Adds
None of the given

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 3 of 10 (Total Marks: 1) Select correct option:
In parser the two LL stand(s) for _____ .
Left-to-right scan of input
left-most derivation
All of the given click here for detail
None of the given

Question # 4 of 10 (Total Marks: 1) Select correct option:
Bottom-up parsers handle a _____ class of grammars.
Large (Page 63)
Small
Medium
None of the given

Question # 5 of 10 (Total Marks: 1) Select correct option:
Compilers are some times classified as.

Single pass
Multi pass
Load and go
All of the given

Question # 6 of 10 (Total Marks: 1) Select correct option:
The Back End of a compiler consist of _____ .

Instruction selection (Page 10)
Register allocation
Instruction scheduling
All of the given

Question # 7 of 10 (Total Marks: 1) Select correct option:
Can a DFA simulate NFA?
Yes click here for detail
No
Sometimes
Depend upon nfa

Question # 8 of 10 (Total Marks: 1) Select correct option:
Which of the statement is true about Regular Languages?
Regular Languages are the most popular for specifying tokens.
Regular Languages are based on simple and useful theory.
Regular Languages are easy to understand.
All of the given (Page 15)

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 9 of 10 (Total Marks: 1) Select correct option:
In a transition table cells of the table contain the _____ state.
Reject state
Next state (Page 18)
Previous state
None of the given

Question # 10 of 10 (Total Marks: 1) Select correct option:
The transition graph for an NFA that recognizes the language (a | b)*abb will have following set of states.
{0}
{0,1}
{0,1,2}
{0,1,2,3} (page 23) Not sure

Question # 1 of 10 (Total Marks: 1) Select correct option:
Functions of Lexical analyzer are?
Removing white space
Removing constants, identifiers and keywords
Removing comments
All of the given click here for detail

Question # 2 of 10 (Total Marks: 1) Select correct option:
_____ method is known as Subset Construction Method.
NFA TO DFA (Page 19)
DFA
DFA maximization
None of given

Question # 3 of 10 (Total Marks: 1) Select correct option:
Which one is NOT the Role of Run-Time System
garbage collection
memory management
run time error checking
none of given (Page 11)

Question # 4 of 10 (Total Marks: 1) Select correct option:
Compiler_____ information from one representation to another
modified
translate (Page 2)
execute
extract

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 5 of 10 (Total Marks: 1) Select correct option:
Front-end of a two pass compiler is consists of Scanner.
True
False (Page 12)

Question # 6 of 10 (Total Marks: 1) Select correct option:
Bottom-up parsing uses only _____ kinds of actions.
Two (Page 64)
Three
Four
Five

Question #7 of 10 (Total Marks: 1) Select correct option:
Bottom-up parsing is also called _____ .
LR parsing (Page 63)
LT parsing
LS parsing
SS parsing

Question # 8 of 10 (Total Marks: 1) Select correct option:
Backtracking is a costly operation which is caused due to Left Recursion.

True
False

Question # 9 of 10 (Total Marks: 1) Select correct option:
A non-recursive predictive parser is also called _____.

table-driven parser (Page 54)
Abstract parser
Conceptual parser
None of the given

Question # 10 of 10 (Total Marks: 1) Select correct option:
AST summarizes the grammatical structure with the details of derivations.
True
False (Page 8)

Question # 1 of 10 (Total Marks: 1) Select correct option:
Backtrack is term associated with Top-down parsing.

True (Page 42)
False

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari

Question # 2 of 10 (Total Marks: 1) Select correct option:
Which of the following statement/s is/are true about Deterministic Finite Automata?

Question # 3 of 10 (Total Marks: 1) Select correct option:
The lexical analyzer partition input string into substrings, called **words**

Question # 4 of 10 (Total Marks: 1) Select correct option:
“” “ab” “abab”... is a string of the language denoted by following Regular Expression:

Muhammad Moaaz Siddiq – MCS(4th)
Moaaz.pk@gmail.com
Campus:- Institute of E-Learning & Moderen Studies (IEMS) Samundari





vulmshelp.com

GRAND QUIZ

LMS handling service are available

PAID

Assignment, QUIZ, GDBZ and GRAND QUIZ SERVICE are available

0304-1659294

0304-1659294

JUNAID MALIK

AL-JUNAID INSTITUTE GROUP

CS606 SOLVED GRAND QUIZZ

1. **The input specification file to flex consists of --------section.**
   a) One
   b) Two
   c) Three
   d) Four
2. **Lexer and scanner are two different phases of compiler**
   a) **True**
   b) **False**
3. **In compiler linear analysis is also called.**
   a) **Lexical analysis**
   b) **Scanning**
   c) **Both lexical analysis and Scanning**
   d) **None of the given**
4. **Which of the following statement is not true**
   a) **The language accepted by finite automata are the language denoted by regular expression.**
   b) **For every DFA there is regular expression denoting its language.**
   c) **For a regular expression, there does exist NFA with L(r) any transit that accepts.**
   d) **None of the given option**
5. **A →B C D**
   B →h B|episilon
   C →C g|G|Ch|i
   D→ A B |episilon
   First of A is----------.
   a) **h, g,i**
   b) **g**
   c) **h**
   d) **none of the given option.**
6. **Top down parsing expends a ---------form the start symbol to the leaves.**

vulmshelp.com

a) Parse tree
b) Abstract syntax tree
c) Parse tree
d) All of the given

7. A top down parse start with the ----------of the parse tree.
a) Root
b) Leaf
c) Middle
d) None of the given

8. Can a DFA simulate NFA ?
a) No
b) Yes
c) Sometimes
d) depends on NFA

9. Bottom up parsing is also called ----------.
a) LR parsing
b) LT parsing
c) LS parsing
d) SS parsing

10. The ----------is optimized for hardware it is to run on.
a) C++ coding
b) C coding
c) Assembly code
d) None

11. Abstract syntax tree summarizes---------- without the detail of derivation.
a) Theory
b) Grammatical structure
c) Data
d) None

12. LR parsing ---------- a string to the start symbol by inverting production.
a) Reduces
b) Shifts

vulmshelp.com

c) Adds
d) None of given

13.Alternative to backtrack in top down parser is----------.
a) Context free grammar
b) Tree
c) Look ahead
d) None of given

14.Typical compilation means programs written in high-level language to low –level----------.
a) Object code
b) Byte code
c) Unicode
d) Both object and byte code

15.The ----------checks the stream of words and their parts of speech for grammatical correctness.
a) Parse
b) Scanner
c) Compiler
d) None of the given

16.A grammar must be----------before use for predictive parsing.
a) Right factored
b) Left factored
c) factored
d) none of the given

17.Parser does not distinguish between valid and invalid sequences of token.
a) True
b) False

18. In compilation process hierarchical analysis is also called------.
a) Parsing
b) Syntax analysis
c) Both Parsing and Syntax analysis
d) None of given

vulmshelp.com

AL-JUNAID INSTITUTE GROUP

19. The pair <role,word> is given the name-----
    a) Word
    b) Token
    c) Syntax
    d) None of the given
20. The back map intermediates(IR) into target
    a) Object code
    b) Machine code
    c) Source code
    d) Linker
21. ----------is evaluated to yield a value.
    a) Command
    b) Expression
    c) Declaration
    d) Declaration and Command
22. Front end of a two pass compiler is consists of scanner.
    a) True
    b) False
23. The ----------returns a sequence of matching token at the output (or an error) and it always returns the longest matching tokens .
    a) Scanner
    b) Lexer
    c) Lexical analysis
    d) All of the given
24. Parser takes token from scanner and tries to generate ---------.
    a) Binary search tree
    b) Parse tree
    c) Syntax trace
    d) None of the given
25. --------avoid hardware stalls and interlock.
    a) Register allocation
    b) Instruction scheduling

vulmshelp.com

c) **Instruction selection**

d) **None of the given**

26.The handle-finding mechanism is the key to efficient----------parsing.

a) **Top-down**

b) **Bottom-up**

c) **Predictive**

d) **None of the given**

27. The input specification file to flex consists of sections:
Three
28. Leaser and scanner are two different phases of compiler.
False
29. In compiler linear analysis is also called.
Both lexical analysis and scanning
30. Which of the following statements is NOT true?
For a regular expression r, there does not exist NFA with L(r) any transit that accept.
31. A --> B C D
B --> h B |episilon
C --> C g |g | C h | i
D --> A B | episilon
First of A is .
h, g, i
32. Top- down parsing expands a from the start symbol to the leaves.
All of the given
33. A top-down parser start with the of the parse tree.
Root
34. Can a DFA simulate NFA?
YES
35. Bottom-up parsing is also called .
LR parsing
36. The is optimized for hardware it is to run on.
Assembly code
37. Abstract syntax tree summarizes without the details of derivation.
Grammatical structure
38. LR parsing a string to the start symbol by inverting productions.
Reduces
39. Alternative to backtrack in top-down parser is .
Look ahead
40. Typical compilation means programs written in high-level languages to low-level .
Object code

vulmshelp.com

41. The checks the stream of words (tokens) and their parts of speech for grammatical correctness.
    Compiler
42. A grammar must be before use for predictive parsing.
    Left factored
43. Parser does not distinguish between valid and invalid sequences of tokens.
    False
44. In compilation process Hierarchical analysis is also called
    Syntax analysis
45. The pair <role, word> is given the name .
    Token
46. The back end maps Intermediate Representation (IR) into target .
    Source code
47. Is evaluated to yield a value.
    Expression
48. Front-end of a two pass compiler is consists of Scanner.
    True
49. The returns a sequence of matching tokens at the output and it always return the longest matching token.
    Lexical analyzer
50. Parser takes tokens from scanner and tries to generate .
    Parse tree
51. Avoid hardware stalls and interlocks.
    Instruction scheduling
52. The handle-finding mechanism is the key to efficient parsing.
    Bottom-up

vulmshelp.com

Compiler Construction (CS606)-

(Covid-19)

Solved by Sonu Ilyas Mughal

Sonu Mughal

03063548328

Contact for Notes And SOLVING Grand Quiz

Question # 1 of 30 (Start time: 03:25:36 PM, 11 July 2020)

The input specification file to flex consists of _____sections:

Select the correct option

○	One
○	Two
■	Three
○	Four

BC140201349: Sidrah Afooj

CS606:Midterm Grand Quiz

Question # 2 of 30 (Start time: 03:26:33 PM, 11 July 2020)

Lexer and scanner are two different phases of compiler

Select the correct option

○	True
●	False

Question # 4 of 30 (Start time: 03:28:55 PM, 11 July 2020)

In compiler linear analysis is also called.

Select the correct option

○	Lexical analysis
○	Scanning
●	Both lexical analysis and scanning
○	None of the given

Question # 5 of 30 (Start time: 03:29:59 PM, 11 July 2020)

Which of the following statements is NOT true?

Select the correct option

○	The language accepted by finite automata are the languages denoted by regular expressions.
○	For every DFA there is a regular expression denoting its language.
●	For a regular expression r, there does not exist NFA with L(r) any transit that accept.
○	None of the given

Question # 6 of 30 (Start time: 03:31:12 PM, 11 July 2020)

A --> B C D
B --> h B | episilon
C --> C g | g | C h | i
D --> A B | episilon

First of A is _____ .

Select the correct option

●	h, g, i
○	g
○	h
○	None of the given.

CS606:Midterm Grand Quiz

Question # 7 of 30 (Start time: 03:32:22 PM, 11 July 2020)

Top-down parsing expands a _____ from the start symbol to the leaves.

Select the correct option

- ○ Parse tree
- ○ Abstract Syntax tree
- ○ Parse tree
- ● All of the given

Question # 9 of 30 (Start time: 03:34:51 PM, 11 July 2020)

A top-down parser starts with the _____ of the parse tree.

Select the correct option

●	Root
○	Leaf
○	Middle
○	None of the given

Question # 10 of 30 (Start time: 03:35:17 PM, 11 July 2020)

Can a DFA simulate NFA?

Select the correct option

○	NO
●	YES
○	Sometimes
○	Depends on NFA

Question # 11 of 30 (Start time: 03:36:07 PM, 11 July 2020)

Bottom-up parsing is also called _____ .

Select the correct option

- (●) LR parsing
- ( ) LT parsing
- ( ) LS parsing
- ( ) SS parsing

Waiting for quiz.vu.edu.pk...

CS606:Midterm Grand Quiz

Question # 12 of 30 (Start time: 03:37:30 PM, 11 July 2020)

The _____ is optimized for hardware it is to run on.

Select the correct option

○	C++ code
○	C code
●	Assembly code
○	None of the given

Question # 13 of 30 (Start time: 03:38:34 PM, 11 July 2020)

Abstract syntax tree summarizes _____without the details of derivation.

Select the correct option

○	Theory
●	Grammatical structure
○	Data
○	None of the give

Question # 14 of 30 (Start time: 03:39:07 PM, 11 July 2020)

LR parsing _____ a string to the start symbol by inverting productions.

Select the correct option

- (●) Reduces
- ( ) Shifts
- ( ) Adds
- ( ) None of the given

Question # 15 of 30 (Start time: 03:39:43 PM, 11 July 2020)

Alternative to backtrack in top-down parser is _____.

Select the correct option

○	Context free grammar
○	Trees
●	Look ahead
○	None of the given

Question # 16 of 30 (Start time: 03:40:46 PM, 11 July 2020)

Typical compilation means programs written in high-level languages to low-level _____.

Select the correct option

●	Object code
○	Byted code
○	Unicode
○	Both Object Code and byte code

Question # 17 of 30 (Start time: 03:41:52 PM, 11 July 2020)

The _____ checks the stream of words (tokens) and their parts of speech for grammatical correctness.

Select the correct option

○	Parser
○	Scanner
●	Compiler
○	None of the given

03063548328

Contact for Notes And SOLVING Grand Quiz

Question # 18 of 30 (Start time: 03:43:20 PM, 11 July 2020)

A grammar must be _____ before use for predictive parsing.

Select the correct option

○	Right factored
●	Left factored
○	Factored
○	None of the given

Question # 19 of 30 (Start time: 03:44:35 PM, 11 July 2020)

Parser does not distinguish between valid and invalid sequences of tokens.

Select the correct option

- ○ True
- ● False

03063548328

Contact for Notes And SOLVING Grand Quiz

Question # 20 of 30 (Start time: 03:45:15 PM, 11 July 2020)

In compilation process Hierarchical analysis is also called

Select the correct option

○	Parsing
●	Syntax analysis
○	Both Parsing and Syntax analysis
○	None of given

Question # 21 of 30 (Start time: 03:46:22 PM, 11 July 2020)

The pair <role,word> is given the name_____.

Select the correct option

○	Word
●	Token
○	Syntax
○	None of the given

Question # 23 of 30 (Start time: 03:48:14 PM, 11 July 2020)

The back end maps Intermediate Representation (IR) into target _____.

Select the correct option

○	Object code
○	Machine code
●	Source code
○	Linker

Question # 24 of 30 (Start time: 03:49:15 PM, 11 July 2020)

_____is evaluated to yield a value.

Select the correct option

- ○ Command
- ● Expression
- ○ Declaration
- ○ Declaration and Command

Question # 25 of 30 (Start time: 03:50:00 PM, 11 July 2020)

Front-end of a two pass compiler is consists of Scanner.

Select the correct option

- (●) True
- ( ) False

Question # 26 of 30 (Start time: 03:51:30 PM, 11 July 2020)

The _____returns a sequence of matching tokens at the output (or an error) and it always return the longest matching token.

Select the correct option

- ○ Scanner
- ○ Lexer
- ● Lexical analyzer
- ○ All of the given

03063548328

Contact for Notes And SOLVING Grand Quiz

Question # 27 of 30 (Start time: 03:52:26 PM, 11 July 2020)

Parser takes tokens from scanner and tries to generate _____ .

Select the correct option

- ○ Binary Search tree
- ● Parse tree
- ○ Syntax trace
- ○ None of the given

Question # 28 of 30 (Start time: 03:53:02 PM, 11 July 2020)

_____avoid hardware stalls and interlocks.

Select the correct option

○	Register allocation
●	Instruction scheduling
○	Instruction selection
○	None of given

Question # 29 of 30 (Start time: 03:53:45 PM, 11 July 2020)

The handle-finding mechanism is the key to efficient _____parsing.

Select the correct option

○	Top-down
●	Bottom-up
○	Predictive
○	None of the given

Sonu Mughal



Contact for Notes And SOLVING Grand Quiz **03063548328**



Contact for Notes And SOLVING Grand Quiz **03063548328**



Vu Mughal

03063548328

Contact for Notes And SOLVING Grand Quiz

CS606 All Midterm Papers Quizez + Mcqz and subjective
Solved by Well wisher

LR parsers can handle _____ grammars.
Left-recursive **Page no: 163**
file-recursive
End-recursive
Start-recursive

_____ convert the reloadable machine code into absolute machine code by linking library and reloadable object files.

Assembler

Loader/link-editor

Compiler

Preprocessor

Consider the grammar A --> B C D

B --> h B | epsilon

C --> C g | g | C h | i

D --> A B | epsilon

Follow of B is _____ .

h

g, h, i, $

g, i

g

Consider the grammar A --> B C D

B --> h B | epsilon

C --> C g | g | C h | i

D --> A B | epsilon

Follow of C is _____ .

g, h, i, $ **Page no : 47**

g, h, $

h, i, $

h, g, $

www.vucybarien.com(well wisher)

An important component of semantic analysis is _____ .
code checking
type checking page no : 6
flush checking
None of the given

In PASCAL _____ represent the inequality test.
:=
=
<>
None of the given

Lexical Analyzer generator _____ is written in Java.

Flex

Jlex Page no : 26

Complex

None of given

_____avoid hardware stalls and interlocks.

Register allocation

Instruction scheduling Page no : 10

Instruction selection

None of given

Consider the following grammar,

A --> B C D

B --> h B | episilon

C --> C g | g | C h | i

D --> A B | episilon

First of A is _____ .
h, g, i
g
h
None of the given

Recursive _____ parsing is done for LL(1) grammar.
Decent Page no : 47
Ascent
Forward
Backward

One of the core tasks of compiler is to generate fast and compact executable code.

www.ucybarien.com(well wisher)

True
False

Left factoring of a grammar is done to save the parser from back tracking.
True Page no:61
False

Responsibility of _____ is to produce fast and compact code.
Instruction selection Page no: 9
Register allocation
Instruction scheduling
None of given

_____ algorithm is used in DFA minimization.
James's
Robert's
Hopcroft's Page no:25
None of given

Compilers are sometimes classified as.

Single pass

Multi pass

Load and go

All of the given

In multi pass compiler during the first pass it gathers information about _____ .
Select correct option:
Declaration
Bindings
Static information
None of the given **

Question # 9 of 10 (Start time: 06:40:30 PM) Total Marks: 1
Flex is an automated tool that is used to get the minimized DFA (scanner).
Select correct option:
True
False

In compilation process Hierarchical analysis is also called
Select correct option:
Parsing
Syntax analysis
Both Parsing and Syntax analysis
None of given

www.vucybrien.com(well wisher)

For each language to make LL(1) grammar, we take two steps, 1st is removing left recurrence and 2nd is applying fin sequence.

True
False

_____is evaluated to yield a value.

Command
Expression
Declaration
Declaration and Command

LR parsers can handle _____ grammars.
Left-recursive **page no : 63**
file-recursive
End-recursive
Start-recursive

Optimal registers allocation is an NP-hard problem.
True
False **Page no : 10**

Parser takes tokens from scanner and tries to generate _____ .

Binary Search tree
Parse tree
Syntax trace **Page no : 6**
None of the given

Front end of two pass compiler takes_____ as input.

Source code **Page no: 5**

Intermediate Representation (IR)

Machine Code

None of the Given

In DFA minimization we construct one _____ for each group of states from the initial DFA.
State **Page no : 25**
NFA
PDA
None of given

In Three-pass compiler _____ is used for code improvement or optimization.

Front End

www.vubaien.com(well wisher)

Middle End **Page no : 10**

Back End

Both Front end and Back end

_____ of a two-pass compiler is consists of Instruction selection, Register allocation and Instruction scheduling.

Back end **Page no : 9**

Front end

Start

None of given

NFA is easy to implement as compared to DFA.

True

False **Page no : 19**

We can get an LL(1) grammar by _____ .

Removing left recurrence

Applying left factoring

Removing left recurrence and Applying left factoring

None of the given

Parser always gives a tree like structure as output

True **page no : 37**

False

Intermediate Representation (IR) stores the value of its operand in

Registers

Memory

Hard disk

Secondary storage

In Back End module of compiler, optimal register allocation uses_____ .

O(log n)

O(n log n)

N P-Complete **Page no : 10**

None of the given

CS 606 Quizez

Can a DFA simulate NFA?

Yes

No

Sometimes

Depend upon nfa

www.vubabrien.com (well wisher)

_____ phase which supports macro substitution and conditional compilation.

Semantic

Syntax

Preprocessing

None

Which of the statement is true about Regular Languages?

Regular Languages are the most popular for specifying tokens.

Regular Languages are based on simple and useful theory.

Regular Languages are easy to understand.

All of the given

Lexer and scanner are two different phases of compiler

True

False Page no :13

Lexical Analyzer generator _____ is written in Java.

Flex

Jlex Page no :26

Complex

None of the given

In a transition table cells of the table contain the _____ state.

Reject state

Next state Page no 18

Previous state

None of the given

The transition graph for an NFA that recognizes the language (a | b)*abb will have following set of states.

{0}

{0,1}

{0,1,2}

{0,1,2,3} not sure

Front end of two pass compiler takes_____ as input.

www.vlcybarien.com(well wisher)

Source code

Intermediate representation

Machine code

None

Functions of Lexical analyzer are?

Removing white space

Removing constants, identifiers and keywords

Removing comments

All of the given

Question # 1 of 10 (Start time: 07:25:59 PM) Total Marks: 1

Front-end of a two pass compiler is consists of Scanner.

Select correct option:

True

False

Question # 2 of 10 (Start time: 07:26:40 PM) Total Marks: 1

LL(1) parsing is called non-predictive parsing.

Select correct option:

True

False

Question # 3 of 10 (Start time: 07:28:09 PM) Total Marks: 1

Recursive _____ parsing is done for LL(1) grammar.

Select correct option:

Backward

Forward

Ascent

Decent

www.vucybarien.com(well wisher)

Question # 4 of 10 (Start time: 07:29:35 PM) Total Marks: 1

In predictive parsing table the rows are _____ .

Select correct option:

Non-terminals

Terminals

Both non-terminal and terminals

None of the given

Question # 5 of 10 (Start time: 07:30:38 PM) Total Marks: 1

We can get an LL(1) grammar by _____ .

Select correct option:

Removing left recurrence

Applying left factoring

Removing left recurrence and Applying left factoring

None of the given

Question # 6 of 10 (Start time: 07:31:48 PM) Total Marks: 1

_____ of a two-pass compiler is consists of Instruction selection, Register allocation and Instruction scheduling.

Select correct option:

Backend

Frontend

Start

www.vucybarien.com(well wisher)

Question # 7 of 10 (Start time: 07:32:26 PM) Total Marks: 1

Consider the grammar

A --> B C D

B --> h B | epsilon

C --> C g | g | C h | i

D --> A B | epsilon First of C is _____ .

Select correct option:

g, I look down at for reference

g

h i

i

Quiz Start Time: 07:25 PM

Time Left 81

sec(s)

Question # 8 of 10 (Start time: 07:33:12 PM) Total Marks: 1

Alternative of the backtrack in parser is Look ahead symbol in _____ .

Select correct option:

input

output

input and output

none

Question # 9 of 10 (Start time: 07:34:09 PM) Total Marks: 1

www.vucybarien.com(well wisher)

AST summarizes the grammatical structure with the details of derivations.

Select correct option:

True

False

Quiz Start Time: 07:25 PM

Time Left 81

sec(s)

Question # 10 of 10 (Start time: 07:35:06 PM) Total Marks: 1

One of the core tasks of compiler is to generate fast and compact executable code.

Select correct option:

True

False

Left factoring is enough to make LL1 grammar

True

False

In LL1() parsing algorithm _____ contains a sequence of grammar symbols.

Stack

Link list

Array

None

Question No 1

What is the role of Automatic Code Generator with respect to compiler?

Answer:

It generates efficient tokens automatically. It is known as Lexer generator.

Question No2

How Linker play an important role with respects to compilers constructions?

Answer:

A linker or link editor is a computer program that takes one or more source files generated by a compiler and combines them into a single executable program.

www.vucybarien.com(well wisher)

Question No3

Consider the grammar for arithmetic expressions?

E→ id Q

Q→ ER| ε

R→ +Q | -Q| *Q| /Q

Show the follow set for all the non-terminal in the grammar

Follow{E}= ?

Solution:

Rules for making follow set:

1. First put $ (the end of input marker) in Follow(S) (S is the start symbol)
2. If there is a production A → aBb, (where a can be a whole string) **then** everything in FIRST(b) except for ε is placed in FOLLOW(B).
3. If there is a production A → aB, **then** everything in FOLLOW(A) is in FOLLOW(B)
4. If there is a production A → aBb, where FIRST(b) contains ε, **then** everything in FOLLOW(A) is in FOLLOW(B)

Here a and b is terminal while A,B are non terminals.

In our question

E→ id Q

Q→ ER| ε

Follow{Q}={+,-,*,/}

R→ +Q | -Q| *Q| /Q

Id, ε, + , -, * , / are terminals

E, Q, R are non terminals.

Follow{E}={+,-,*,/,$}

www.vucybarien.com(well wisher)

Question No 4

Consider the following grammar. Calculate the first set of non- terminal S, A and B

S⟶ AB

A⟶ a | ε

B⟶ b | ε

Solution:

S⟶ AB

Rules for making first set:

1) If S→ A….Z then first of S will be First of A….Z
2) If First of A….Z all contains ε then first of S will also contain ε
3) If any (single) First set from A….Z $_{doesn't}$ contain ε in their First set then First of S will also doesn't contain ε.
4) The first of A will be first non terminal on r.h.s
5) The first of B will also be the non terminal on r.h.s
6) Note: ε is also a terminal.

Now make first sets for S,A and B

First{A}={a, ε }

First{B}={b ,ε}

Question NO: 5

Consider the grammar

A → B C D

B → h B | ε

C → C g | g | C h | i

D → A B |ε

Answer:

Rules for making first set:

1) If S→ A….Z then first of S will be First of A….Z
2) If First of A….Z all contains ε then first of S will also contain ε
3) If any (single) First set from A….Z $_{doesn't}$ contain ε in their First set then First of S will also doesn't contain ε.

www.vucybarien.com(well wisher)

4) The first of A will be first non terminal on r.h.s
5) The first of B will also be the non terminal on r.h.s
6) Note: ε is also a terminal.

A → B C D

B → h B | ε

C → C g | g | C h | i

D → A B |ε

First {A}={h,g,i}

Note: As the production C didn't contain ε so we didn't add ε in First {A}.

First {B}={h, ε}

First{C}={g,i}

First{D}={ h,g,i, ε }

Note: **We added** ε there because D originally has ε as a terminal in it.

Q. In two pass assembler what is the objective of First Pass? 2 marks

Answer:

First pass contains Scanner + Parser in it. Scanner takes the streams of chcracters as input and converts them into words or tokens

<Token type, word>

While Parser check those tokens for syntactic analysis and semantic analysis if found errors then report them after that it develop an IR.

Q. What is Register allocation? 3 marks

Answer:

Registers in a CPU play an important role for providing high speed access to operands. Memory access is an order of magnitude slower than register access. The back end attempts to have each operand value in a register when it is used. However, the back end has to manage a limited set of resources when it comes to the register file. The number of registers is small and some registers are pre-allocated for specialize used, e.g., program counter, and thus are not available for use to the back end. Optimal register allocation is NP-Complete.

Q. Up frontier discovery useful for top down parser.

www.rgpvbanen.com(well wisher)

Answer:

A bottom-up parser builds the parse tree starting with its leaves and working toward its root. The upper edge of this partially constructed parse tree is called its upper frontier

Parser tree 3 marks

Answer:

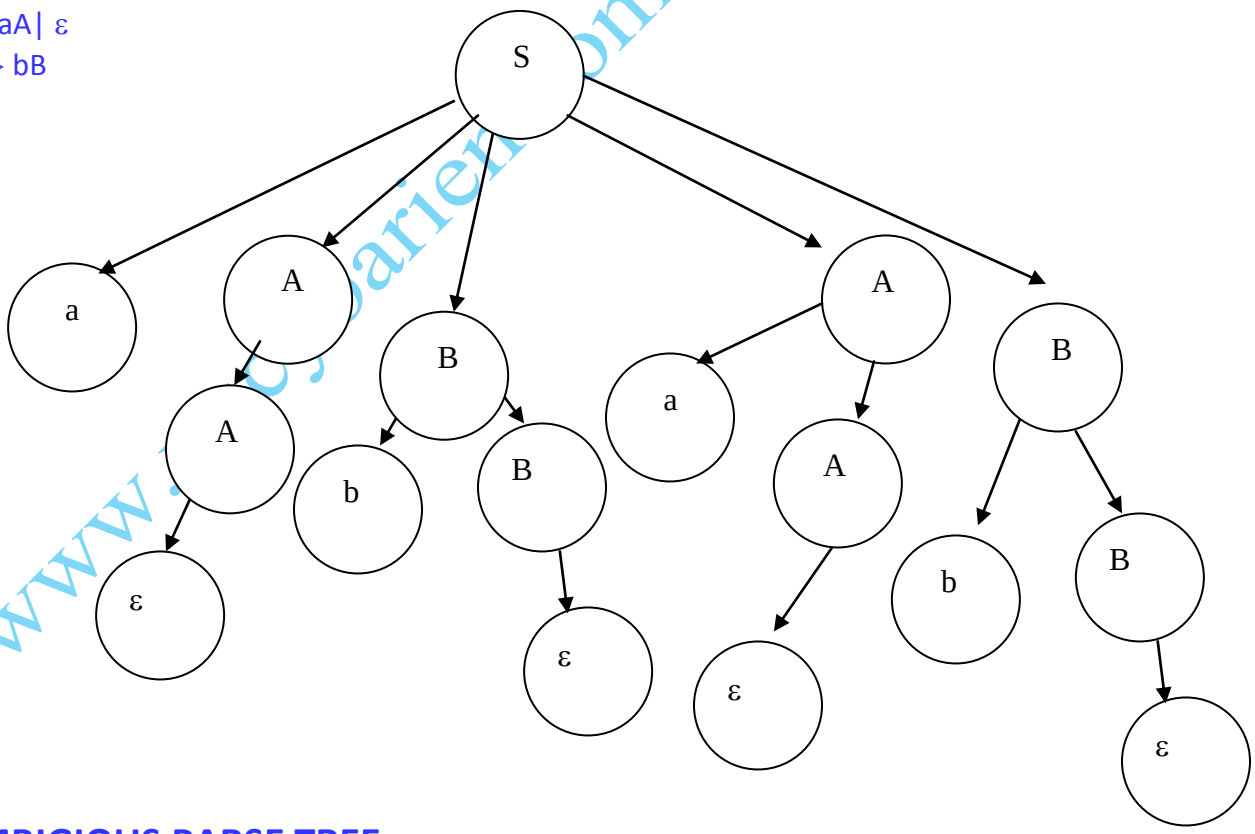
A parse can be represented by a tree: parse tree or syntax tree.

Q1)Considering string "ab", show that the following grammar is ambiguous using parse trees. The ε below is epsilon.

S → A B A B
A → a A | ε
B → b B | ε

Solution:

S→ABAB
A→aA| ε
B–> bB



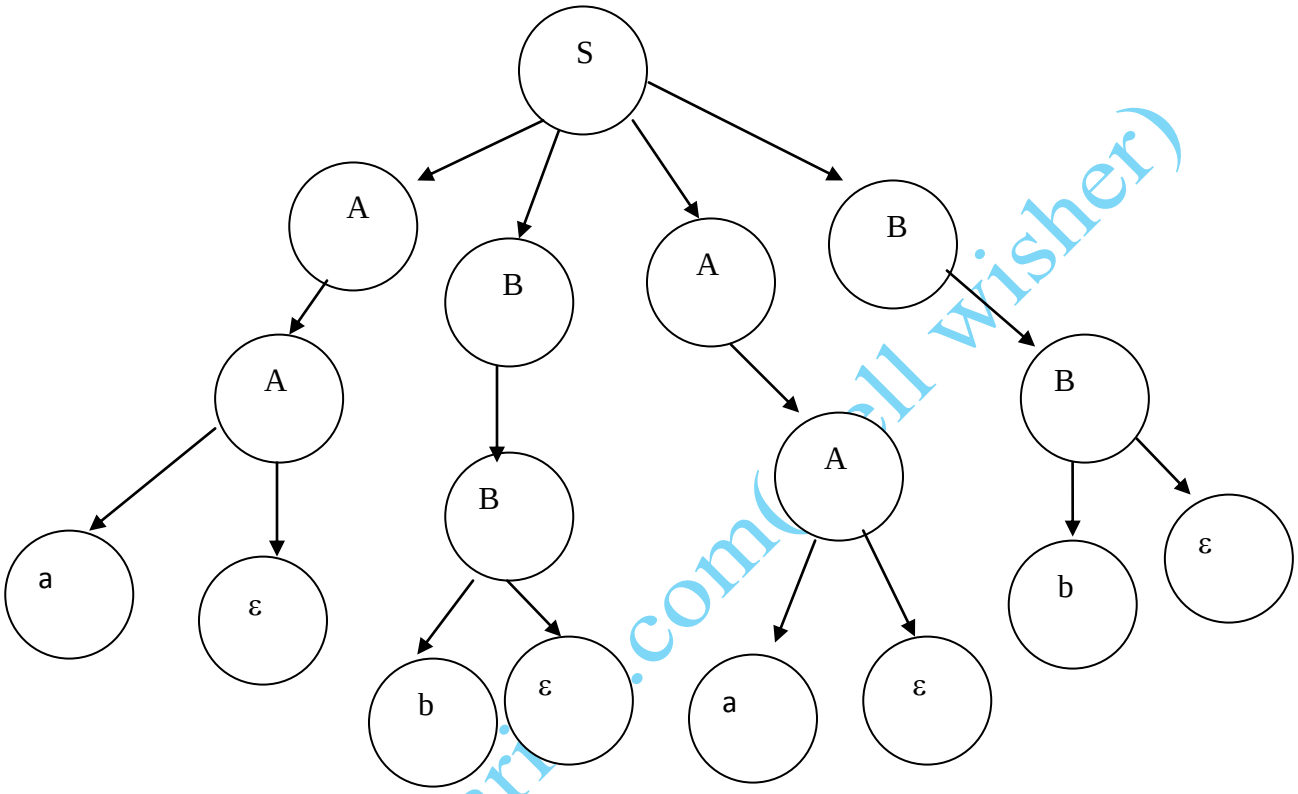
www.[illegible]om(well wisher)

AMBIGIOUS PARSE TREE:

Can be written as

S→ABAB
A→ a ε
B--> b ε

Here it create ambiguity the string “ab” can be re written with internal A



www.vucybari.com(Well wisher)

Θ2) vConsider the following grammar;

Statement → if expression then statement else statement
Statement → if expression then statement

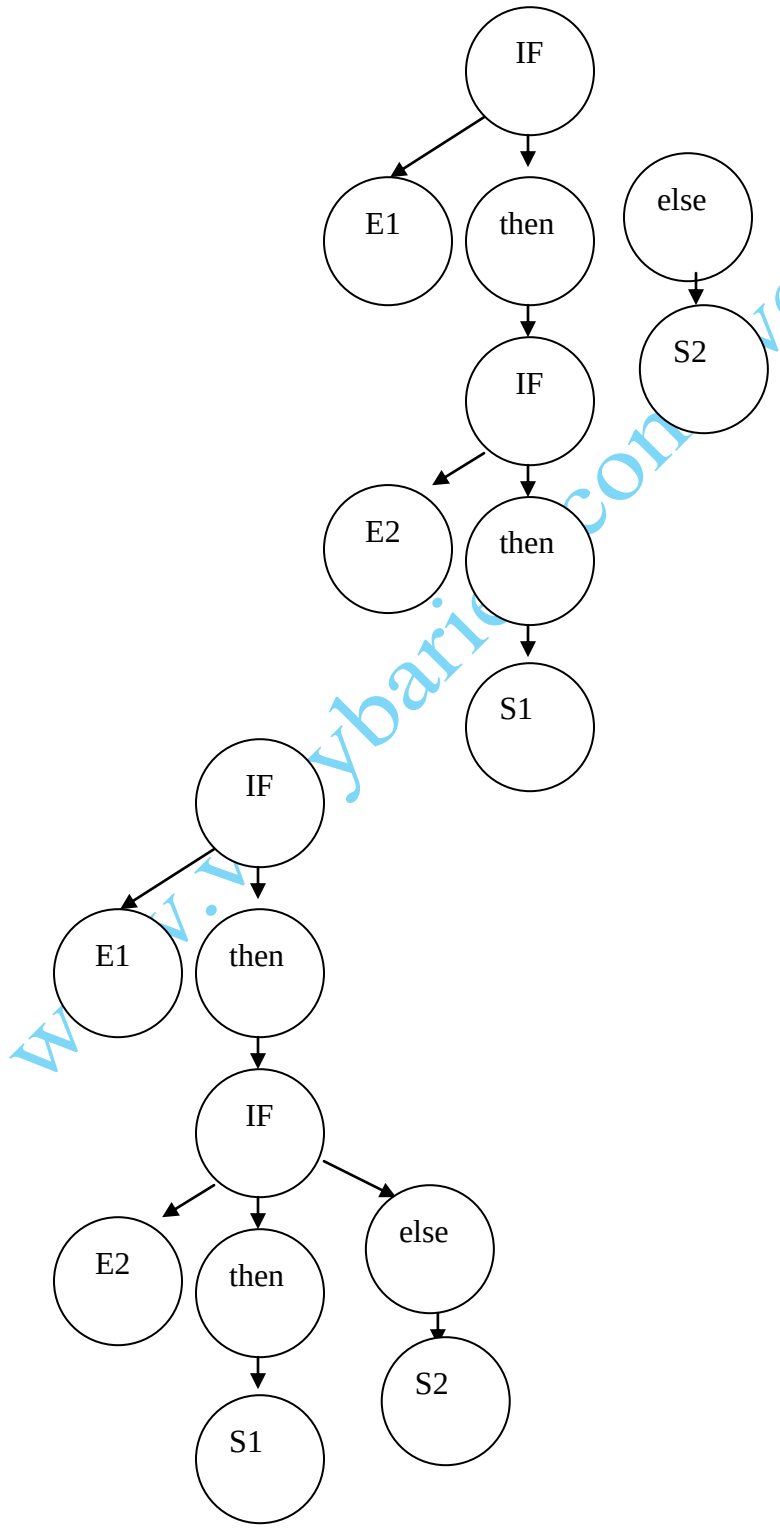
You are required to provide an alternate production(s) so that it may become free from the backtracking.
(5Marks)
Solution:

Statement → if expression then statement

Statement → if expression then statement else statement

Solution:

IF E1 then IF E2 then S1 Else S2



www.vybarie.com (well wisher)

This is the ambiguous situation where

1)

IF E1
Then
  IF E2
  S1
Else
  S2

2)

We can derivate this grammar in another way also

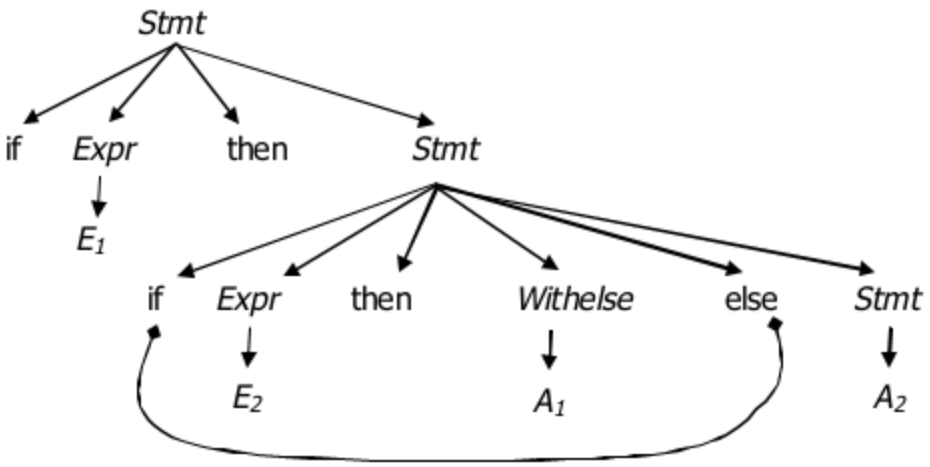
IF E1
Then
  IF E2
    S1
  Else
    S2

Here to correct it out we introduce another grammar for same situation by including one more NT.

Stmnt→ if E1 then stmnt
  | If E1 then with else else stmnt
  | Assignment
Stmnt→ if E2 then with else else stmnt
  | assignment

IF E1
Then
  IF E2
  A1
  Else
  A2

www.vucybarien.com(well wisher)



Q4) How recursive descent parser is implemented using any Object Oriented language?

(3Marks)

Answer:

We can use C++ to code the recursive descent parser in an object oriented manner. We associate a C++ class with each non-terminal symbol. An instantiated object of a non-terminal class contains pointer to the parse tree.

Q6) Write a regular expression for the language of all words that starts and ends with different letters.

(2 Marks)

Answer:

a(a+b)*b+b(a+b)*a

www.vucybarien.com (well wisher)



Compiler Construction(CS606)- Lecture Handouts

Lecture 1 5
- Course Organization 5
- Why Take this Course 6
- What are Compilers 6
- Typical Compilation 7
- Issues in Compilation 8

Lecture 2 9
- Two-pass Compiler 9
- Front End 10
- Parser 11

Lecture 3 13
- Abstract Syntax Trees 13
- The Back End 14

Lecture 4 15
- Register Allocation 15
- Instruction Scheduling 15
- Three-pass Compiler. 15
- Role of Run-time System 16

Lecture 5 17
- Lexical Analysis 17

Lecture 6 20
- How to Describe Tokens? 20

Lecture 7 23
- Table Encoding of FA 23

Lecture 8 27
- NFA ? DFA Construction 27

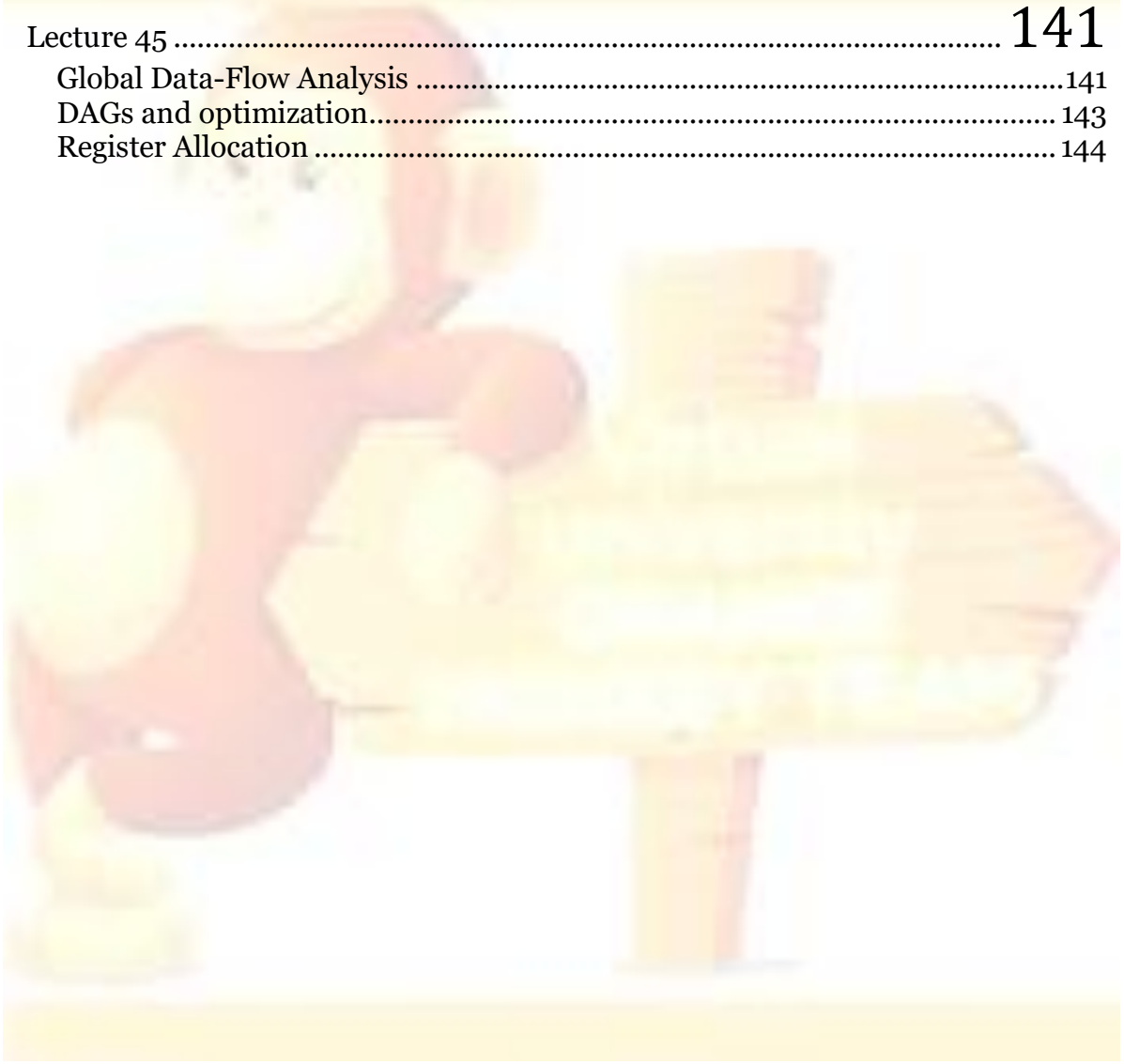
Lecture 9 30
- DFA Minimization 30
- Lexical Analyzers 30

Lecture 10 33
- Running the Scanner 33
- Parsing 39

Lecture 11 41
Syntactic Analysis 41
Semantic Analysis 41
Lecture 12 44
Parse Trees 44
Ambiguous Grammars 46
Context-Free Grammars 48
Parsing Techniques 49
Lecture 13 50
Top-Down Parser 50
Lecture 14 54
The LL(1) Property 54
Lecture 15 58
Lecture 16 60
Non-recursive Predictive Parsing 61
LL(1) Parsing Algorithm 62
Lecture 17 64
LL(1) Table Construction 64
Lecture 18 65
FOLLOW Sets 66
Lecture 19 68
Lecture 20 70
Bottom-up Parsing 70
Shift-Reduce Parsing 71
Lecture 21 73
Lecture 22 75
Lecture 23 77
Handles 77
LR(1) Parsers 78
Lecture 24 80
Canonical Collection 80
The Closure Procedure 80
Lecture 25 82

Lecture 26 84
Lecture 27 87
LR Table Construction 87
Lecture 28 92
LR(1) Skeleton Parser 92
Shift/Reduce Conflicts 92
Lecture 29 94
Lecture 30 96
Parser Generators 96
YACC Parser Generator 96
Lecture 31 98
Beyond Syntax 98
Attribute Grammars 98
Lecture 32 102
Evaluation Methods 102
Lecture 33 104
Lecture 34 105
Intermediate Representations 107
Lecture 35 108
IR Taxonomy 108
Graphical IRs 108
Linear IRs 110
Lecture 36 111
Assignment Statement 111
Lecture 37 112
Representing Linear Codes 112
Flow-of-Control Statements 113
Lecture 38 115
Three-Address Statement Types 115
Lecture 39 117
Boolean Expressions 117
Lecture 40 122
Lecture 41 125

Lecture 42 129
Code Generation 129
Lecture 43 132
Control Flow Graph - CFG 132
Basic Block Code Generation 136
Computing Live/Next Use Information 136
Lecture 44 137
Lecture 45 141
Global Data-Flow Analysis 141
DAGs and optimization 143
Register Allocation 144



Lecture 1

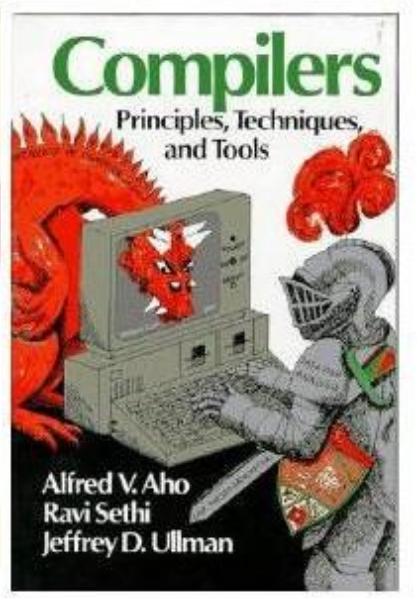
Course Organization

The course is organized around theory and significant amount of practice. The practice will be in the form of home works and a project. The project is the highlight of the course: you will build a full compiler for subset of Java- like language. The implementation will in C++ and you will generate Intel x86 assembly language code. The project will be done in six parts; each will be a programming assignment.

The grade distribution will be

Theory	Homeworks	10%
	Exams	50%
Practice	Project	40%

The primary text for the course is *Compilers – Principles, Techniques and Tools* by Aho, Sethi and Ullman. This is also called the Dragon Book; here is the image on the cover of the book:



Why Take this Course

There are number of reason for why you should take this course. Let's go through a few

Reason #1: understand compilers and languages

We all have used one or computer languages to write programs. We have used compilers to "compile" our code and eventually turn it into an executable. While we worry about data structures, algorithms and all the functionality that our application is supposed to provide, we perhaps overlook the programming language, the structure of the code and the language semantics. In this course, we will attempt to understand the code structure, understand language semantics and understand relation between source code and generated machine code. This will allow you to become a better programmer

Reason #2: nice balance of theory and practice

We have studied a lot of theory in various CS courses related to languages and grammar. We have covered mathematical models: regular expressions, automata, grammars and graph algorithms that use these models. We will now have an opportunity to put this theory into practice by building a real compiler.

Reason #3: programming experience

Creating a compiler entails writing a large computer program which manipulates complex data structures and implement sophisticated algorithm. In the process, we will learn more about C++ and Intel x86 assembly language. The experience, however, will be applicable if we desire to use another programming language, say Java, and generate code for architecture other than Intel.

What are Compilers

Compilers translate information from one represent ation to another. Thus, a tool that translates, say, Russian into English could be labeled as a compiler. In this course, however, information = program in a computer language. In this context, we will talk of compilers such as Typical Compiler Examples: C++ Builder, Microsoft Visual Studio Express,Microsoft Visual C++ 2012, VBCC.. Application that convert, for example, a Word file to PDF or PDF to Postscript will be called "translators". In this course we will study typical compilation: from programs written in high- level languages to low- level object code and machine code.

Typical Compilation

Consider the source code of C function

```
int expr( int n )
{
        int d;
        d = 4*n*n*(n+1)*(n+1); return
        d;
}
```

Expressing an algorithm or a task in C is optimized for human readability and comprehension. This presentation matches human notions of grammar of a programming language. The function uses named constructs such as variables and procedures which aid human readability. Now consider the assembly code that the C compiler gcc generates for the Intel platform:

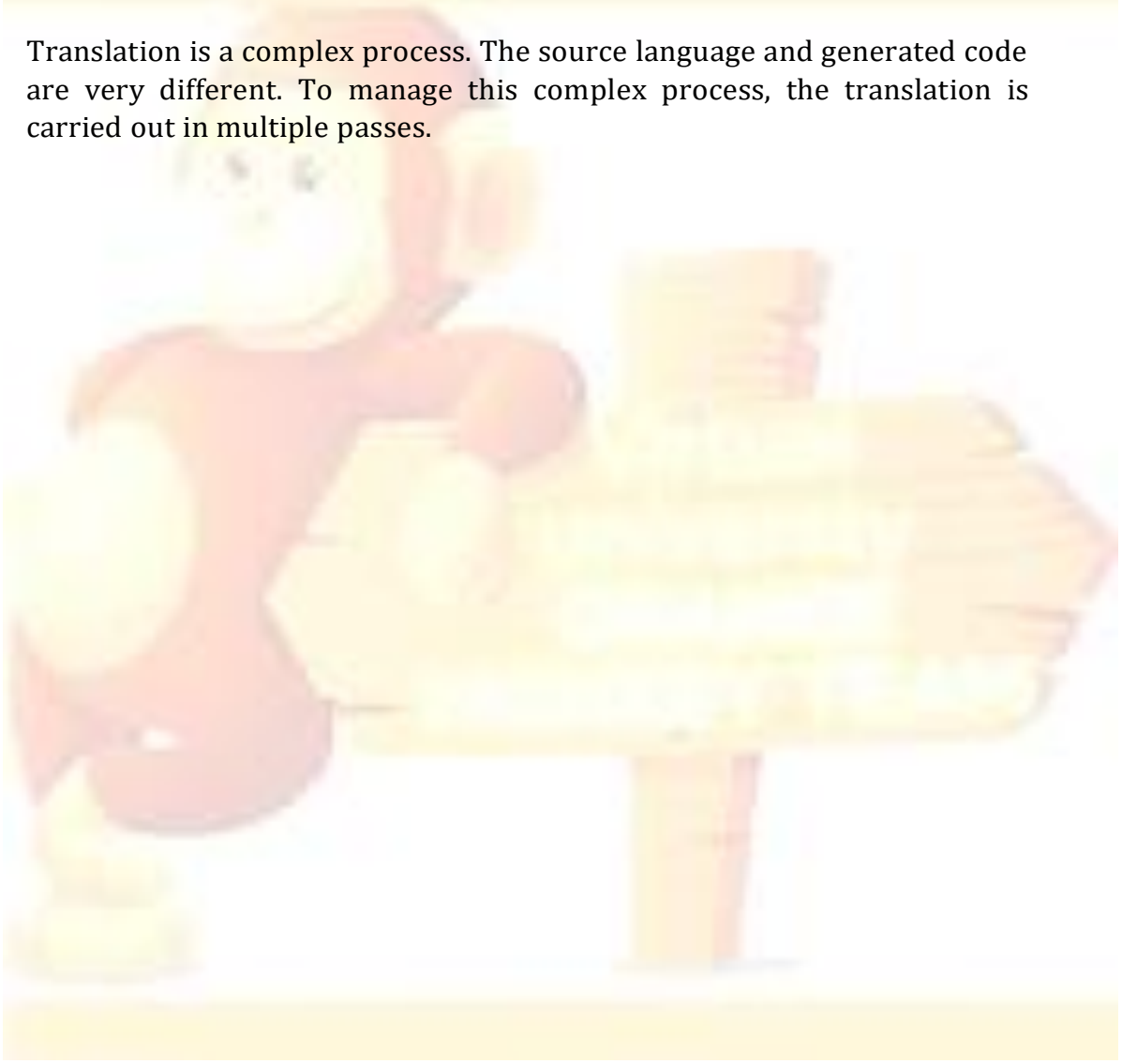
```
.globl _expr
_expr:
        pushl %ebp movl
        %esp,%ebp subl
        $24,%esp
        movl 8(%ebp),%eax
        movl %eax,%edx
        leal 0(,%edx,4),%eax movl
        %eax,%edx imull
        8(%ebp),%edx movl
        8(%ebp),%eax incl %eax
        imull %eax,%edx movl
        8(%ebp),%eax incl
        %eax
        imull %eax,%edx movl
        %edx,-4(%ebp) movl -
        4(%ebp),%edx movl
        %edx,%eax
        jmp L2
        .align 4
L2:
        leave
        ret
```

The assembly code is optimized for hardware it is to run on. The code consists of machine instructions, uses registers and unnamed memory locations. This version is much harder to understand by humans

Issues in Compilation

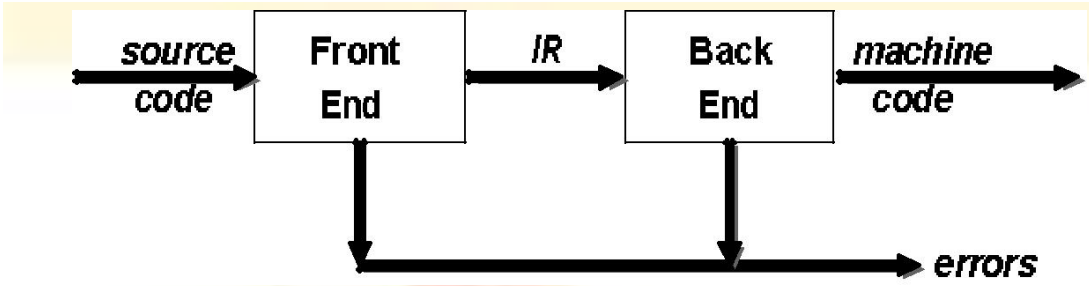
The translation of code from some human readable form to machine code must be "correct", i.e., the generated machine code must execute precisely the same computation as the source code. In general, there is no unique translation from source language to a destination language. No algorithm exists for an "ideal translation".

Translation is a complex process. The source language and generated code are very different. To manage this complex process, the translation is carried out in multiple passes.



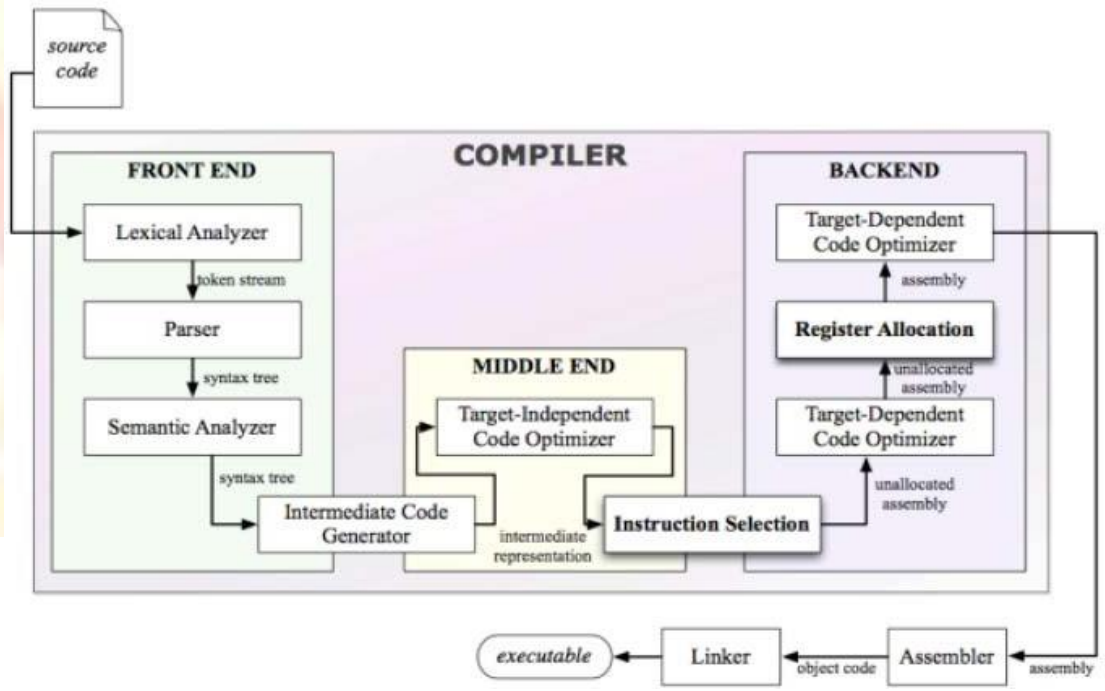
Lecture 2

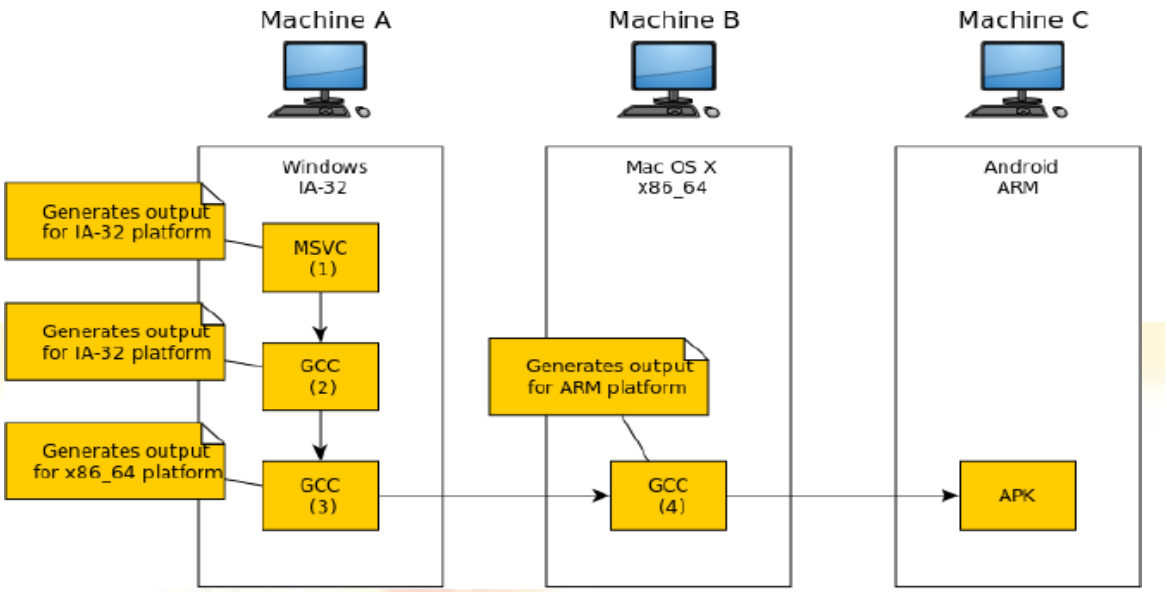
Two-pass Compiler



The figure above shows the structure of a two-pass compiler. The front end maps legal source code into an intermediate representation (IR). The back end maps IR into target machine code. An immediate advantage of this scheme is that it admits multiple front ends and multiple passes.

The algorithms employed in the front end have polynomial time complexity while majority of those in the backend are NP-complete. This makes compiler writing a challenging task.

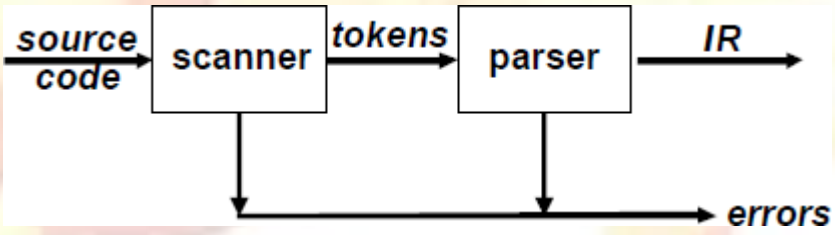




Picture of cross compile like GCC for compilation of C code and generating code for multiple platforms.

Let us look at the details of the front and back ends.

Front End



The front end recognizes legal and illegal programs presented to it. When it encounters errors, it attempts to report errors in a useful way. For legal programs, front end produces IR and preliminary storage map for the various data structures declared in the program. The front end consists of two modules:

1. Scanner
2. Parser

Scanner

The scanner takes a program as input and maps the character stream into "words" that are the basic unit of syntax. It produces pairs – a word and its part

of speech. For example, the input

x = x + y

becomes

<id,**x**>
<assign,**=**
> <id,**x**>
<op,**+**>
<id,**y**>

We call the pair "<token type, word>" a *token*. Typical tokens are: *number*, *identifier*, +, -, *new*, *while*, *if*.

Parser

The parser takes in the stream of tokens, recognizes context- free syntax and reports errors. It guides context-sensitive ("semantic") analysis for tasks like type checking. The parser builds IR for source program.

The syntax of most programming languages is specified using Context-Free Grammars (CFG). Context- free syntax is specified with a grammar *G*=(*S,N,T,P*) where

- *S* is the *start* symbol
- *N* is a set of *non-terminal* symbols
- *T* is set of *terminal* symbols or words
- *P* is a set of *productions* or rewrite rules

For example, the Context-Free Grammar for arithmetic expressions is

1.	*goal*	?	*expr*
2.	*expr*	?	*expr op term*
3.		|	*term*
4.	*term*	?	number
5.		|	id
6.	*op*	?	+
7.		|	–

For this CFG,

S = *goal*
T = { number, id, +, -}
N = {*goal*, *expr*, *term*, *op*}
P = { 1, 2, 3, 4, 5, 6, 7}

Given a CFG, we can *derive* sentences by repeated substitution. Consider the sentence

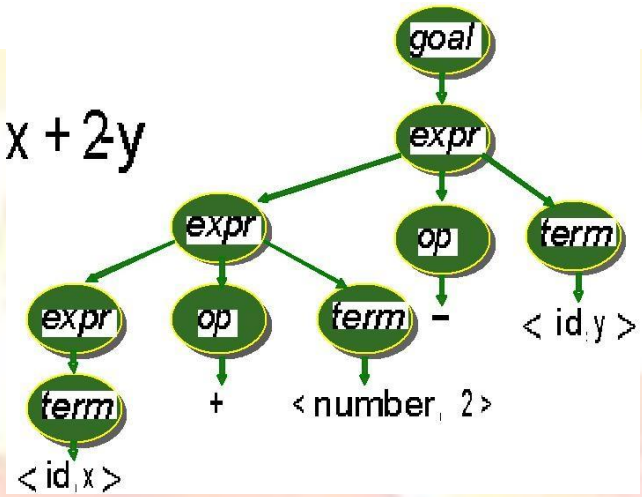
x + 2 – y

Production	***Result***
	goal
1: *goal* → *expr*	*expr*
2: *expr* → *expr op term*	*expr op term*
5: *term* → id	*expr op* y
7: *op* → –	*expr* – y
2: *expr* → *expr op term*	*expr op term* – y
4: *term* → number	*expr op* 2 – y
6: *op* → +	*expr* + 2 – y
3: *expr* → *term*	*term* + 2 – y
5: *term* → id	x + 2 – y

To recognize a valid sentence in some CFG, we *reverse* this process and build up a *parse*, thus the name "parser".

Lecture 3

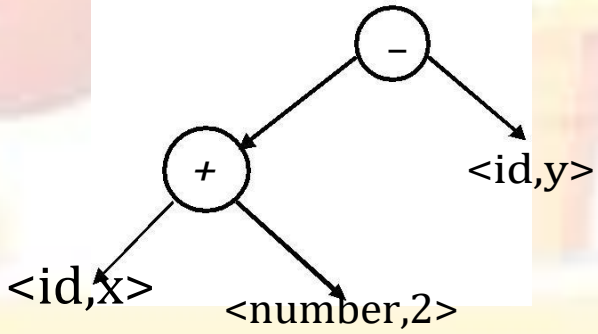
A parse can be represented by a tree: *parse tree* or *syntax tree*. For example, here is the parse tree for the expression x+2-y



The parse tree captures all rewrite during the derivation. The derivation can be extracted by starting at the root of the tree and working towards the leaf nodes.

Abstract Syntax Trees

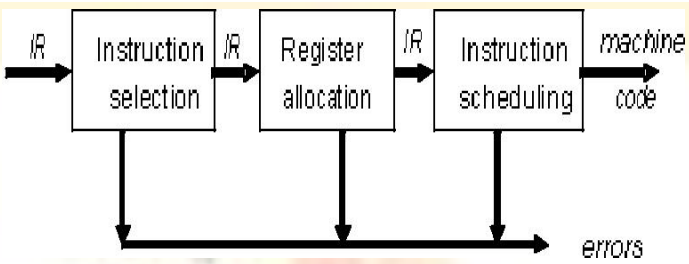
The parse tree contains a lot of unneeded information. Compilers often use an *abstract syntax tree (AST)*. For example, the AST for the above parse tree is



This is much more concise; AST summarizes grammatical structure without the details of derivation. ASTs are one kind of *intermediate representation (IR)*.

The Back End

The back end of the compiler translates IR into target machine code. It chooses machine (assembly) instructions to implement each IR operation. The back end ensure conformance with system interfaces. It decides which values to keep in registers in order to avoid memory access; memory access is far slower than register access.



The back end is responsible for instruction selection so as to produce fast and compact code. Modern processors have a rich instruction set. The back end takes advantage of target features such as addressing modes. Usually, instruction selection is viewed as a pattern matching problem that can be solved by dynamic programming based algorithms. Instruction selection in compilers was spurred by the advent of the VAX-11 which had a CISC (Complex Instruction Set Computer) architecture. The VAX-11 had a large instruction set that the compiler could work with.

Lecture 4

CISC architecture provided a rich set of instructions and addressing modes but it made the job of the compiler harder when it came to generate efficient machine code. The RISC architecture simplified this problem.

Register Allocation

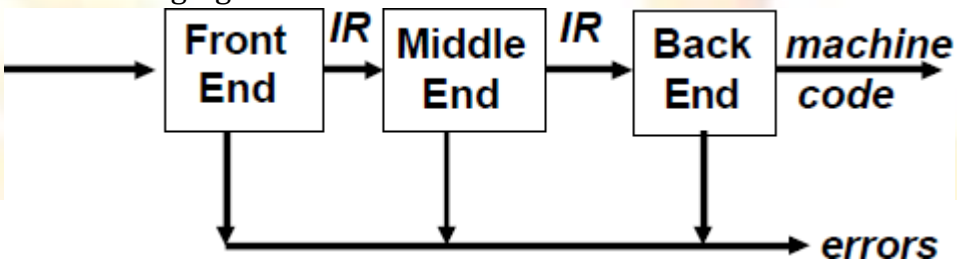
Registers in a CPU play an important role for providing high speed access to operands. Memory access is an order of magnitude slower than register access. The back end attempts to have each operand value in a register when it is used. However, the back end has to manage a limited set of resources when it comes to the register file. The number of registers is small and some registers are pre-allocated for specialize used, e.g., program counter, and thus are not available for use to the back end. Optimal register allocation is *NP-Complete*.

Instruction Scheduling

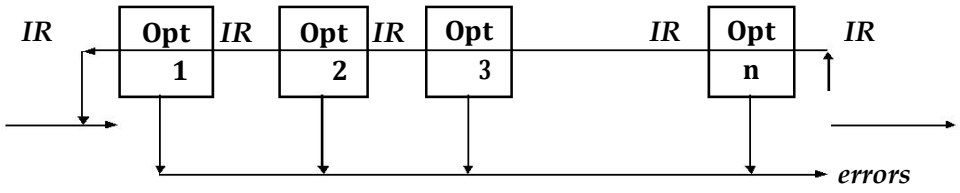
Modern processors have multiple functional units. The back end needs to schedule instructions to avoid hardware stalls and interlocks. The generated code should use all functional units productively. Optimal scheduling is *NP-Complete* in nearly all cases.

Three-pass Compiler

There is yet another opportunity to produce efficient translation: most modern compilers contain three stages. An intermediate stage is used for code improvement or optimization. The topology of a three-pass compiler is shown in the following figure:



The middle end analyzes IR and rewrites (or *transforms*) IR. Its primary goal is to reduce running time of the compiled code. This may also improve space usage, power consumption, etc. The middle end is generally termed the "Optimizer". Modern optimizers are structured as a series of passes:



Typical transformations performed by the optimizer are:

- Discover & propagate some constant value
- Move a computation to a less frequently executed place
- Specialize some computation based on context
- Discover a redundant computation & remove it
- Remove useless or unreachable code
- Encode an idiom in some particularly efficient form

Role of Run-time System

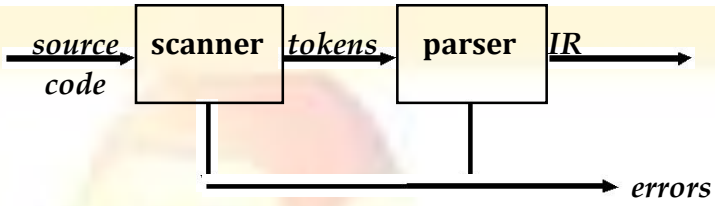
The executable code typically runs as a process in an Operating System Environment. The application will need a number of resources from the OS. For example, dynamic memory allocation and input output. The process may spawn more processes of threads. If the underline architecture has multiple processors, the application may want to use them. Processes communicate with other and may share resources. Compilers need to have an intimate knowledge of the runtime system to make effective use of the runtime environment and machine resources. The issues in this context are:

- Memory management
- Allocate and de-allocate memory
- Garbage collection
- Run-time type checking
- Error/exception processing
- Interface to OS – I/O
- Support for parallelism
- Parallel threads
- Communication and synchronization

Lecture 5

Lexical Analysis

The scanner is the first component of the front-end of a compiler; parser is the second



The task of the scanner is to take a program written in some programming language as a stream of characters and break it into a stream of tokens. This activity is called *lexical analysis*. A token, however, contains more than just the words extracted from the input. The lexical analyzer partition input string into substrings, called *words*, and classifies them according to their role.

Tokens

A token is a *syntactic category* in a sentence of a language. Consider the sentence:

He wrote the program

of the natural language English. The words in the sentence are: “He”, “wrote”, “the” and “program”. The blanks between words have been ignored. These words are classified as subject, verb, object etc. These are the roles. Similarly, the sentence in a programming language like C:

```
if(b == 0) a = b
```

the words are “if”, “(”, “b”, “==”, “0”, “)”, “a”, “=” and “b”. The roles are keyword, variable, boolean operator, assignment operator. The pair <role,word> is given the name *token*. Here are some familiar tokens in programming languages:

< Identifiers: x y11 maxsize
< Keywords: if else while for
< Integers: 2 1000 -445L
< Floats: 2.0 0.0034 1e5
< Symbols: ( ) + * / { } < > ==
< Strings: “enter x” “error”

Ad-hoc Lexer

The task of writing a scanner is fairly straight forward. We can hand-write code to generate tokens. We do this by partitioning the input string by reading left-to-right, recognizing one token at a time. We will need to look-ahead in order to decide where one token ends and the next token begins. The following C++ code present is template for a Lexer class. An object of this class can produce the desired tokens from the input stream.

```
class Lexer
{
        Inputstream s;
        char next; //look ahead
        Lexer(Inputstream  _s)
        {
                s = _s;
                next = s.read();
        }

        Token nextToken() {
                if( idChar(next) )return readId(); if( number(next)
                )return readNumber(); if( next == "" ) return
                readString();
                …
                …
        }
        Token readId() { string id =
                ""; while(true){
                        char c = input.read();
                        if(idChar(c) == false)
                                return new Token(TID,id); id = id
                        + string(c);
                }
        }
        boolean idChar(char c)
        {
                if( isAlpha(c) ) return true; if( isDigit(c)
                ) return true; if( c == '_' ) return true;

                return false;
        }
        Token readNumber(){ string
                num = "";
                while(true){
```

```
                next = input.read(); if(
                !isNumber(next))
                        return new Token(TNUM,num); num
        = num+string(next);
    }
}
```

This works ok, however, there are some problem that we need to tackle.

- We do not know what kind of token we are going to read from seeing first character.
- If token begins with "i", is it an identifier "i" or keyword "if"?
- If token begins with "=", is it "=" or "=="?

We can extend the Lexer class but there are a number of other issues that can make the task of hand-writing a lexer tedious. We need a more principled approach. The most frequently used approach is to use a *lexer generator* that generates efficient tokenizer *automatically*.

Lecture 6

How to Describe Tokens?

Regular Languages are the most popular for specifying tokens because

- These are based on simple and useful theory,
- Are easy to understand and
- Efficient implementations exist for generating lexical analyzers based on such languages.

Languages

Let ∑ be a set of characters. ∑ is called the *alphabet*. A *language over* ∑ is set of strings of characters drawn from ∑. Here are some examples of languages:

- Alphabet=English characters
- Language=English sentences
- Alphabet = ASCII
  Language = C++, Java, C# programs

Languages are sets of strings (finite sequence of characters). We need some notation for specifying which sets we want. For lexical analysis we care about *regular languages*. Regular languages can be described using *regular expressions*. Each regular expression is a notation for a regular language (a set of words). If **A** is a regular expression, we write **L(A)** to refer to language denoted by **A**.

Regular Expression

A *regular expression* (*RE*) is define as

a	ordinary character from ∑
ε	the empty string
R|S	either R or S
RS	R followed by S (concatenation)
R*	concatenation of R zero or more times (R* = ε |R|RR|RRR...)

Regular expression extensions are used as convenient notation of complex RE:

R?	ε | R (zero or one R)
R^+	RR* (one or more R)

(R) R (grouping)
[abc] a|b|c (any of listed)
[a-z] a|b|....|z (range)
[^ab] c|d|....(anything but 'a''b')

Here are some Regular Expressions and the strings of the language denoted by the RE.

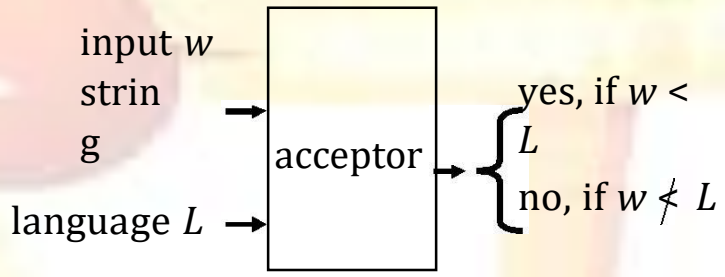
RE	Strings in L(R)
a	"a"
ab	"ab"
a|b	"a" "b"
(ab)*	"" "ab" "abab" ...
(a| ε)b	"ab" "b"

Here are examples of common tokens found in programming languages.

digit	'0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
integer	digit digit*
identifier	[a-zA- Z_][a- zA-Z0-9_]*

Finite Automaton

We need mechanism to determine if an input string *w* belongs to *L(R),* the language denoted by regular expression *R*. Such a mechanism is called an *acceptor*.

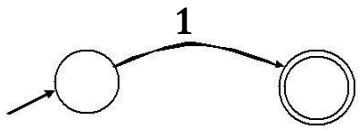


The acceptor is based on Finite Automata (FA). A *Finite Automaton* consists of

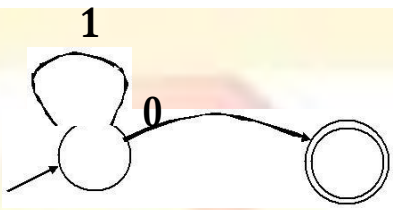
- An input alphabet ∑
- A set of states
- A start (initial) state
- A set of transitions
- A set of accepting (final) states

A finite automaton *accepts* a string if we can follow transitions labeled with characters in the string from start state to some accepting state. Here are some examples of FA.

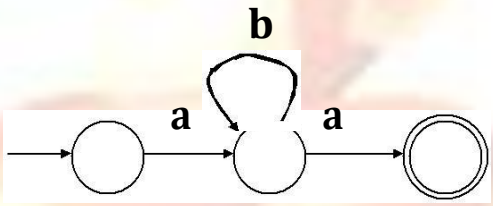
- A FA that accepts only “1”



- A FA that accepts any number of 1’s followed by a single 0



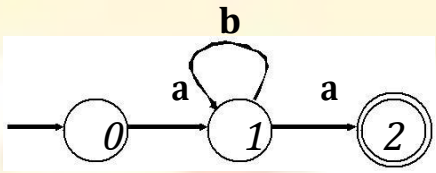
- A FA that accepts ab*a (∑: {a,b})



Lecture 7

Table Encoding of FA

A FA can be encoded as a table. This is called a *transition table*. The following example shows a FA encoded as a table.



	a	b
0	*1*	**err**
1	*2*	*1*
2	**err**	**err**

The rows correspond to states. The characters of the alphabet set < appear in columns. The cells of the table contain the next state. This encoding makes the implementation of the FA simple and efficient. It is equally simple to simulate or run the FA given an alphabet and a string of the language and its associated alphabet set ∑. The C++ code shows such a FA simulator.

```
int trans_table[NSTATES][NCHARS]; int
accept_states[NSTATES];
int state = INITIAL;
while(state != err){
        c = input.read(); if(c == EOF
        ) break;
        state=trans_table[state][c];
}
return accept_states[state];
```

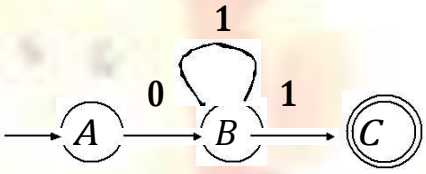
RE ? Finite Automata

We now have a strategy for building lexical analyzer. The tokens we want to recognize are encoded using regular expressions. If we can build a FA for regular expressions, we have our lexical analyzer. The question is can we build

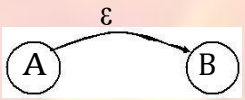
a finite automaton for every regular expression? The answer, fortunately, is yes – build FA inductively based on the definition of Regular Expression (RE). The actual algorithm actually builds N*ondeterministic Finite Automaton* (NFA) from RE. Each RE is converted into an NFA. NFA are joined together with ε -moves. The eventual NFA is then converted into a *Deterministic Finite Automaton* (DFA) which can be encoded as a transition table. Let us discuss how this happens.

Nondeterministic Finite Automaton (NFA)

An NFA can have multiple transitions for one input in a given state. In the following NFA, and input of 1 can cause the automaton to go to state B or C.



It can also have e–moves; the automaton machine can move from state A to state B without consuming input.



The operation of the automaton is not completely defined by input. A NFA can choose whether to make ε-moves and which of multiple transitions to take for a single input. The acceptance of NFA for a given string is achieved if it *can* get in a final state.

Deterministic Finite Automaton (NFA)

In Deterministic Finite Automata (DFA), on the other hand, there is only one transition per input per state. There are no ε -moves. Upon execution of the automaton, a DFA can take *only one path* through the state graph and is therefore completely determined by input.

NFAs and DFAs recognize the same set of languages (regular languages). DFAs are easier to implement – table driven. For a given language, the NFA can be simpler than the DFA. DFA can be exponentially larger than NFA. NFAs are the key to automating RE ? DFA construction.

RE ? NFA Construction

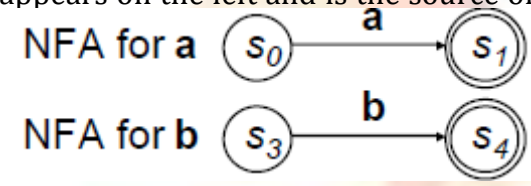
The algorithm for RE to DFA conversion is called *Thompson's Construction*. The algorithm appeared in *CACM* 1968. The algorithm builds an NFA for each RE

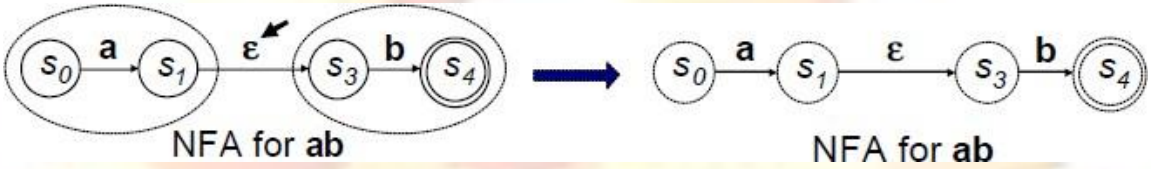
term. The NFAs are then combined using ε -moves. The NFA is converted into a DFA using the subset construction procedure. The number of states in the resulting DFA are minimized using the *Hopcroft's algorithm*.

Given a RE, we first create NFA pattern for each symbol and each operator. We then join them with ε -moves in precedence order. Here are examples of such constructions:

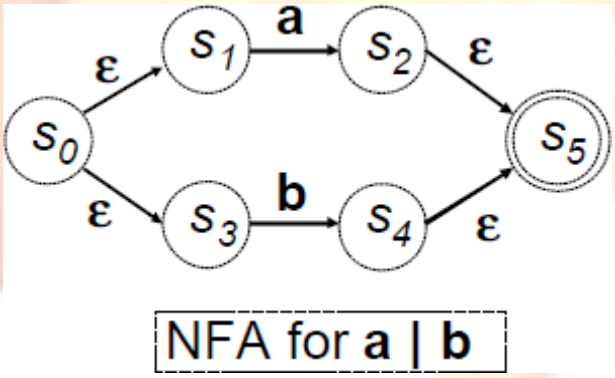
1. NFA for RE **ab.**

The following figures show the construction steps. NFAs for RE **a** and RE **b** are made. These two are combined using an ε -move; the NFA for RE **a** appears on the left and is the source of the ε -transition.

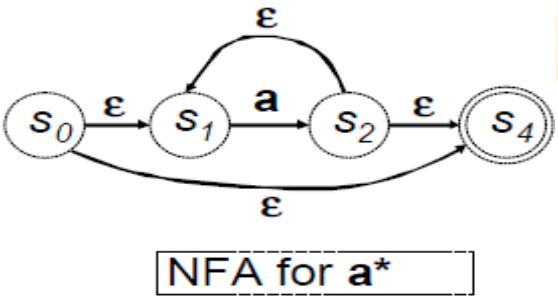




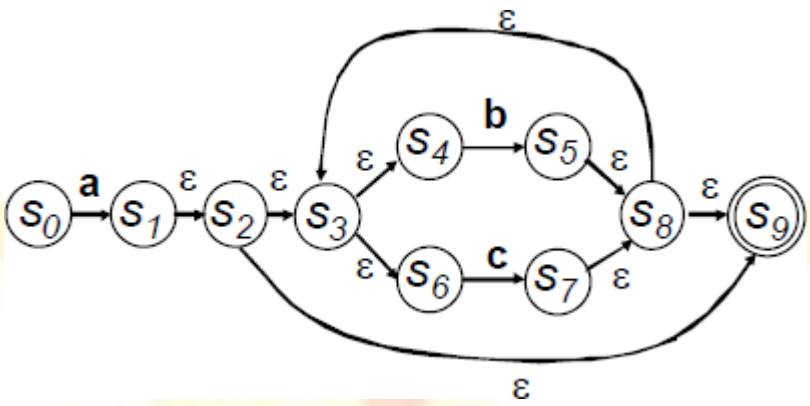
2. NFA for RE **a|b**



3. NFA for RE **a***



3. NFA for RE **a (b|c)***



Lecture 8

NFA ? DFA Construction

The algorithm is called *subset construction*. In the transition table of an NFA, each entry is a set of states. In DFA, each entry is a single state. The general idea behind NFA-to-DFA construction is that each DFA state corresponds to a set of NFA states. The DFA uses its state to keep track of all possible states the NFA can be in after reading each input symbol.

We will use the following operations.

- *ε-closure(T)*:
  set of NFA states reachable from some NFA state *s* in *T* on ? -transitions alone.
- *move(T,a)*:
  set of NFA states to which there is a transition on input *a* from some NFA state *s* in set of states *T*.

Before it sees the first input symbol, NFA can be in any of the state in the set ε-*closure(s0)*, where *s0* is the start state of the NFA. Suppose that exactly the states in set *T* are reachable from *s0* on a given sequence of input symbols. Let *a* be the next input symbol. On seeing *a*, the NFA can move to any of the states in the set *move(T,a)*. Let *a* be the next input symbol. On seeing *a*, the NFA can move to any of the states in the set *move(T,a)*. When we allow for ε-transitions, NFA can be in any of the states in ε-*closure(move(T,a))* after seeing *a*.

Subset Construction

Algorithm:

Input: NFA N with state set S, alphabet ∑, start state s_0, final states F *Output:* DFA D with state set S', alphabet ∑, start states, s_0' = ε -closure(s_0), final states F' and transition table: S' x ∑ ? S'

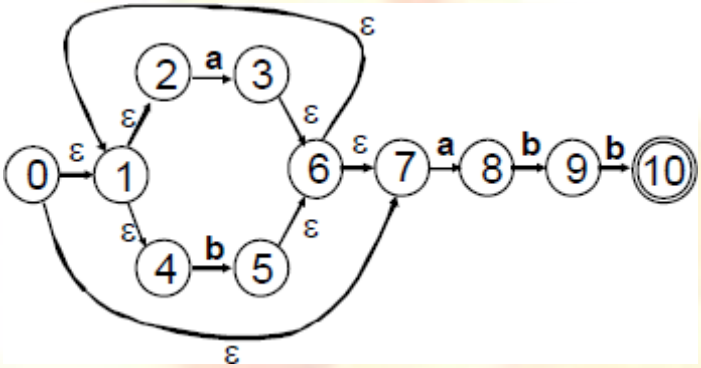
// initially, e-closure(s_0) is the only state in D states S' and it is unmarked *s0'* = ε -closure(*s0*)
S' = {*s0'*} (unmarked)

while (there is some unmarked state *T* in *S')* mark state *T*
for all *a* in ∑ *do*
U = ? ε-closure(move(*T*,*a*));
if U not already in *S'*

add U as an unmarked state
to *S'* Dtran(*T*,*a*) = *U*;
end for
end while
F':
for each DFA state *S*
if S contains an NFA final state
mark *S* as DFA final
state
end algorithm

Example

Let us apply the algorithm to the NFA for **(a | b)*abb.** ∑ is {**a**,**b**}.



The start state of equivalent DFA is ε -*closure(0)*, which is *A* = {0,1,2,4,7}; these are exactly the states reachable from state 0 via ε -transition. The algorithm tells us to mark A and then compute ε -*closure(move(A,a))*

move(A,a)), is the set of states of NFA that have transition on '**a**' from members of *A*. Only 2 and 7 have such transition, to 3 and 8. So, ε -closure(move(*A*,**a**)) = ε -closure({3,8}) = {1,2,3,4,6,7,8}. Let *B* = {1,2,3,4,6,7,8}; thus *Dtran*[*A*,**a**] = *B*

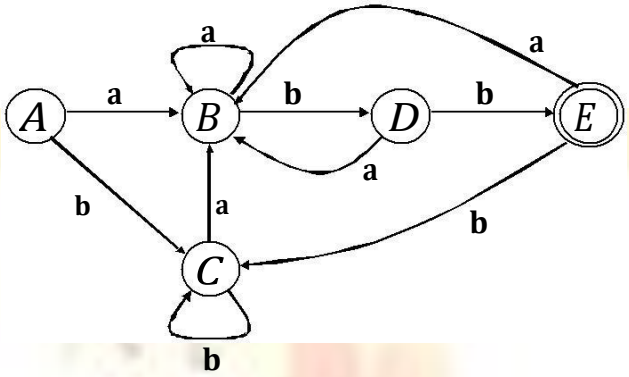
For input **b**, among states in *A*, only 4 has transition on **b** to 5. Let C = ε -closure({5}) = {1,2,4,5,6,7}. Thus, *Dtran*[*A*,**b**] = *C*

We continue this process with the unmarked sets B and C, i.e., ε -closure(*move*(*B*,**a**)), ε -closure(*move*(*B*,**b**)), ε -closure(*move*(*C*,**a**)) and ε -closure(*move*(*C*,**b**)) until all sets and states of DFA are marked. This is certain since there are *only* 2^{11} (!) different subsets of a set of 11 states. A set, once marked, is marked forever. Eventually, the 5 sets are:

1. *A*={0,1,2,4,7}
2. *B*={1,2,3,4,6,7,8}
3. *C*={1,2,4,5,6,7}
4. *D*={1,2,4,5,6,7,9}
5. *E*={1,2,4,5,6,7,10}

A is start state because it contains state 0 and E is the accepting state because it contains state 10.

The subset construction finally yields the following DFA



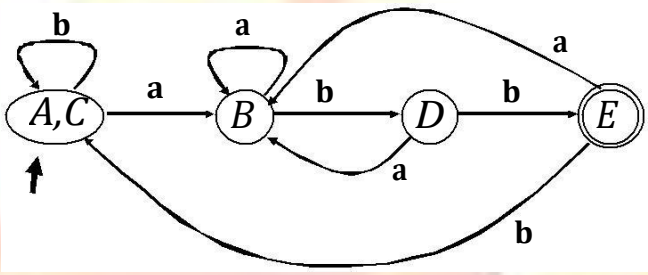
The Resulting DFA can be encoded as the following transition table

State	Input symbol	
	a	**b**
A	*B*	*C*
B	*B*	*D*
C	*B*	*C*
D	*B*	*E*
E	*B*	*C*

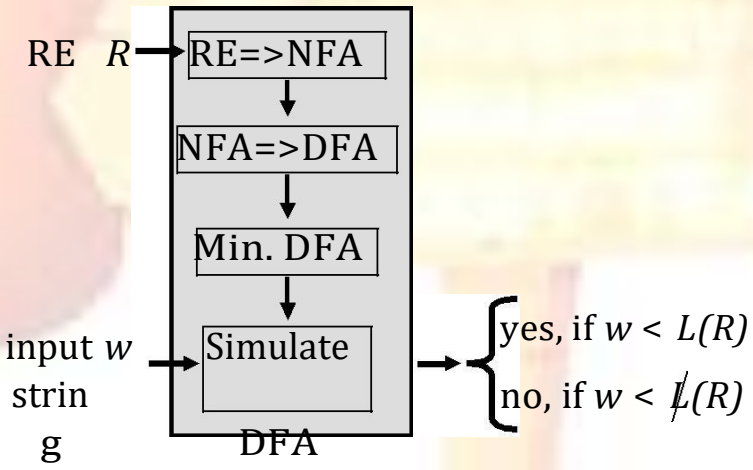
Lecture 9

DFA Minimization

The generated DFA may have a large number of states. The Hopcroft's algorithm can be used to minimize DFA states. The behind the algorithm is to find groups of *equivalent states.* All transitions from states in one group G_1 go to states in the same group G_2. Construct the minimized DFA such that there is one state for each group of states from the initial DFA. Here is the minimized version of the DFA created earlier; states A and C have been merged.



We can construct an optimized acceptor with the following structure:



Lexical Analyzers

Lexical analyzers (scanners) use the same mechanism but they have multiple RE descriptions for multiple tokens and have a character stream at the input. The lexical analyzer returns a sequence of matching tokens at the output (or an error) and it always return the longest matching token.

Lexical Analyzer Generators

The process of constructing a lexical analyzer can be automated. We only need to specify Regular expressions for tokens and rules for assigning priorities for multiple longest match cases, e.g, "==" and "=", "==" is longer.

Two popular lexical analyzer generators are

- **Flex :** generates lexical analyzer in C or C++. It is more modern version of the original Lex tool that was part of the AT&T Bell Labs version of Unix.
- **Jlex:** written in Java. Generates lexical analyzer in Java

Using Flex

We will use for the projects in this course. To use Flex, one has to provide a specification file as input to Flex. Flex reads this file and produces an output file contains the lexical analyzer source in C or C++.

The input specification file consists of three
sections: C or C++ and flex definitions
%%
token definitions and actions
%%
user code

The symbols "%%" mark each section. A detailed guide to Flex is included in supplementary reading material for this course. We will go through a simple example.

The following is the Flex specification file for recognizing tokens found in a C++ function. The file is named "lex.l"; it is customary to use the ".l" extension for Flex input files.

```
%{
#include "tokdefs.h"
%}
D               [0-9]
L               [a-zA-Z_]
id      {L}({L}|{D})*
%%
"void" {return(TOK_VOID);} "int"
{return(TOK_INT);}
"if"        {return(TOK_IF);}
```

Specification File lex.l"else"
{return(TOK_ELSE);}
"while"{return(TOK_WHILE)}; "<="
{return(TOK_LE);}
">=" {return(TOK_GE);} "=="
{return(TOK_EQ);} "!="
{return(TOK_NE);} {D}+
{return(TOK_INT);} {id}
{return(TOK_ID);} [\n]|[\t]|[ ] ;
%%

The file lex.l includes another file named “tokdefs.h”. The content of tokdefs.h are

```
#define TOK_VOID 1
#define TOK_INT 2
#define TOK_IF 3
#define TOK_ELSE 4
#define TOK_WHILE 5
#define TOK_LE 6
#define TOK_GE 7
#define TOK_EQ 8
#define TOK_NE 9
#define TOK_INT 10
#define TOK_ID 111
```

Flex creates C++ classes that implement the lexical analyzer. The code for these classes is placed in the Flex’s output file. Here, for example, is the code needed to invoke the scanner; this is placed in main.cpp:

```
void main()
{
        FlexLexer lex;
        int tc = lex.yylex(); while(tc
        != 0) {
                cout << tc << “,” <<lex.YYText() << endl; tc =
                lex.yylex();
        }
}
```

The following commands can be used to generate a scanner executable file in windows.

```
flex lex.l g++ –c
lex.cpp
g++ –c main.cpp
g++ –o lex.exe lex.o main.o
```

Lecture 10

Running the Scanner

Here is the output of the scanner when executed and given the file main.cpp as input, i.e., the scanner is being asked to provide tokens found in the file main.cpp:

lex <main.cpp

```
259,void
258,main
283,(
284,)
285,{
258,FlexLexer
258,lex
290,;
260,int
258,tc
266,=
258,lex
291,.
258,yylex
283,(
284,)
290,;
263,while
283,(
258,tc
276,!=
257,0
284,)
258,cout
279,<<
258,tc
279,<<
292,","
279,<<
258,lex
291,.
258,YYText
283,(
284,)
279,<<
```

```
258,endl
290,;
258,tc
266,=
258,lex
291,.
258,yylex
283,(
284,)
290,;
286,}
```

Flex input for C++

As an illustration of the power of Flex, here is the input for generating scanner for C++ compiler.

```
/*
 *  ISO C++ lexical analyzer.
 *  Based on the ISO C++ draft standard of December '96.
 */

%{
#include <ctype.h>
#include  <stdio.h>
#include tokdefs.h"

int lineno;

static int yywrap(void);
static void skip_until_eol(void);
static void skip_comment(void);
static int check_identifier(const char *); %}

intsuffix       ([uU][lL]?)|([lL][uU]?)
fracconst       ([0-9]*\.[0-9]+)|([0-9]+\.)
exppar
t                       [eE][-+]?[0-9]+
floatsuffix     [fFlL]
chartext        ([^'])|(\\.)
stringtext      ([^"])|(\\.)
%%
%%
"\n"        { ++lineno; }
[\t\f\v\r ]+        { /* Ignore whitespace. */ }
"/*"        { skip_comment(); }
"//"        { skip_until_eol(); }
"{"         { return '{'; }
```

```
"<%"      { return '{'; }
"}"       { return '}'; }
"%>"      { return '}'; }
"["       { return '['; }
"<:"      { return '['; }
"]"       { return ']'; }
":>"      { return ']'; }
"("       { return '('; }
")"       { return ')'; }
";"       { return ';'; }
":"       { return ':'; }
"..."       { return ELLIPSIS; }
"?"       { return '?'; }
"::"      { return COLONCOLON; }
"."       { return '.'; }
".*" { return DOTSTAR; } "+" { return '+'; }
"-" { return '-'; } "*" { return '*'; }
"/" { return '/'; }
"%" { return '%'; }
"^" { return '^'; }
"xor" { return '^'; }
"&" { return '&'; }
"bitand" { return '&'; }
"|" { return '|'; }
"bitor" { return '|'; }
"~" { return '~'; }
"compl" { return '~'; }
"!" { return '!'; }
"not" { return '!'; }
"=" { return '='; }
"<" { return '<'; }
">" { return '>'; }
"+=" { return ADDEQ; }"-
=" { return SUBEQ; }
"*=" { return MULEQ; }
"/=" { return DIVEQ; }
"%=" { return MODEQ; }
"^=" { return XOREQ; }
"xor_eq" { return XOREQ; }
"&=" { return ANDEQ; }
"and_eq" { return ANDEQ; }
"|=" { return OREQ; }
"or_eq" { return OREQ; }
"<<" { return SL; }
">>" { return SR; }
"<<=" { return SLEQ; }
">>=" { return SREQ; }
"==" { return EQ; }
"!=" { return NOTEQ; }
```

```
"not_eq" { return NOTEQ; }
"<=" { return LTEQ; }
">=" { return GTEQ; }
"&&" { return ANDAND; }
"and" { return ANDAND; }
"||" { return OROR; }
"or" { return OROR; }
"++" { return PLUSPLUS; }
"--" { return MINUSMINUS; }
"," { return ','; }
"->*" { return ARROWSTAR; }
"->" { return ARROW; }
"asm" { return ASM; }
"auto" { return AUTO; }
"bool" { return BOOL; }
"break" { return BREAK; }
"case" { return CASE; }
"catch" { return CATCH; }
"char" { return CHAR; }
"class" { return CLASS; }
"const" { return CONST; }
"const_cast" { return CONST_CAST; }
"continue" { return CONTINUE; }
"default" { return DEFAULT; }
"delete" { return DELETE; }
"do" { return DO; }
"double" { return DOUBLE; }
"dynamic_cast" { return DYNAMIC_CAST; }
"else" { return ELSE; }
"enum" { return ENUM; }
"explicit" { return EXPLICIT; }
"export" { return EXPORT; }
"extern" { return EXTERN; }
"false" { return FALSE; }
"float" { return FLOAT; }
"for" { return FOR; }
"friend" { return FRIEND; }
"goto" { return GOTO; }
"if" { return IF; }
"inline" { return INLINE; }
"int" { return INT; }
"long" { return LONG; }
"mutable" { return MUTABLE; }
"namespace" { return NAMESPACE; }
"new" { return NEW; }
"operator" { return OPERATOR; }
```

```
"private" { return PRIVATE; }
"protected" { return PROTECTED; }
"public" { return PUBLIC; }
"register" { return REGISTER; }
"reinterpret_cast" { return REINTERPRET_CAST; }
"return" { return RETURN; }
"short"  {  return  SHORT;  }
"signed"  {  return  SIGNED;  }
"sizeof"  {  return  SIZEOF;  }
"static" { return STATIC; }
"static_cast" { return STATIC_CAST; }
"struct" { return STRUCT; }
"switch" { return SWITCH; }
"template" { return TEMPLATE; }
"this" { return THIS; }
"throw" { return THROW; }
"true" { return TRUE; }
"try" { return TRY; }
"typedef" { return TYPEDEF; }
"typeid" { return TYPEID; }
"typename" { return TYPENAME; }
"union" { return UNION; }
"unsigned" { return UNSIGNED; }
"using" { return USING; }
"virtual" { return VIRTUAL; }
"void" { return VOID; }
"volatile" { return VOLATILE; }
"wchar_t" { return WCHAR_T; }
"while" { return WHILE; }
[a-zA-Z_][a-zA-Z_0-9]*
{ return check_identifier(yytext); }

"0"[xX][0-9a-fA-F]+{intsuffix}? { return INTEGER; }
"0"[0-7]+{intsuffix}? { return INTEGER; }
[0-9]+{intsuffix}? { return INTEGER; }

{fracconst}{exppart}?{floatsuffix}? { return FLOATING; }
[0-9]+{exppart}{floatsuffix}? { return FLOATING; }

"'"{chartext}*"'" { return CHARACTER; }
"L'"{chartext}*"'" { return CHARACTER;
}

"\""{stringtext}*"\""          { return STRING; }
"L\""{stringtext}*"\""         { return STRING; }
.          { fprintf(stderr,
   "%d: unexpected character `%c'\n", lineno,
```

```
        yytext[0]); }
%%
static int
yywrap(void)
{
        return 1;
}
static void
skip_comment(vo
id)
{
        int c1, c2;

        c1 = input();
        c2 = input();

        while(c2 != EOF && !(c1 == '*' && c2 == '/'))
        {
                if (c1 == '\n')
                        ++lineno
                        ;
                c1 = c2;
                c2 = input();
        }
}
static void
skip_until_eol(void
)
{
        int c;

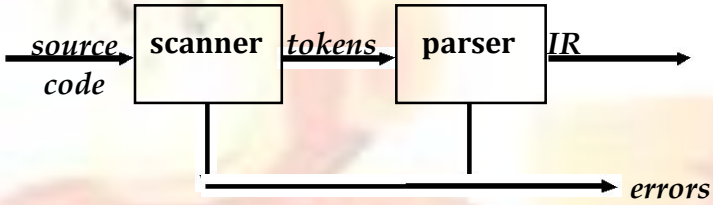
        while ((c = input()) != EOF && c != '\n')
                ;
        ++lineno;
}

static int check_identifier(const
char *s)
{
        /*
         *  This function should check if `s' is a
         *  typedef name or a class
         *  name, or a enum name, ... etc. or
         *  an identifier.
         */
        switch (s[0]) {
```

```
            case 'D': return
            TYPEDEF_NAME;  case  'N':
            return NAMESPACE_NAME;
            case 'C': return CLASS_NAME;
            case 'E': return ENUM_NAME;
            case  'T': return  TEMPLATE_NAME;
            }
            return IDENTIFIER;
}
```

Parsing

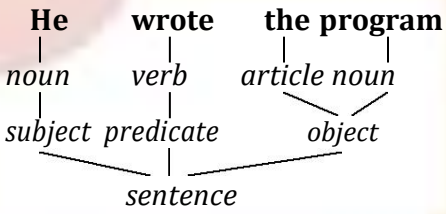
We now move the second module of the front-end: the parser. Recall the front-end components:



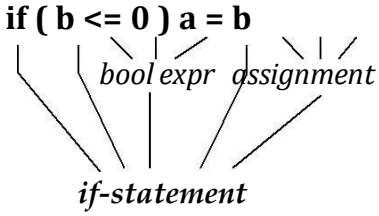
The parser checks the stream of words (tokens) and their parts of speech for grammatical correctness. It determines if the input is *syntactically* well formed. It guides context-sensitive ("semantic") analysis (type checking). Finally, it builds IR for source program.

Syntactic Analysis

Consider the sentence "He wrote the program". The structure of the sentence can be described using grammar syntax of English language.



The analogy can be carried over to syntax of sentences in a programming language. For example, an if-statement has the syntax



The parser ensures that sentences of a programming language that make up a program abide by the syntax of the language. If there are errors, the parser will detect them and reports them accordingly. Consider the following code segment that contains a number of syntax errors:

```
int* foo(int i, int j))
{
  for(k=0; i j; ) fi( i > j )
        return j;
}
```

It is clear that a scanner based upon regular expressions will not be able to detect syntax error.



Lecture 11

Syntactic Analysis

Consider the following C++ function. There are a number of syntax errors present.

```
1.      int* foo(int i, int j))
2.      {
3.              for(k=0; i j; )
4.                      fi( i > j )
5.              return j;
6.      }
```

Line 1 has extra parenthesis at the end. The boolean expression in the for loop in line 3 is incorrect. Line 4 has a missing semicolon at the end. All such errors are due to the fact the function does not abide by the syntax of the C++ language grammar.

Semantic Analysis

Consider the English language sentence "He wrote the computer". The sentence is syntactically correct but semantically wrong. The meaning of the sentence is incorrect; one does not "write" a computer. Issues related to meaning fall under the heading of *semantic analysis*. The following C++ function has semantic errors. The type of the local variable **sum** has not been declared. The returned value does not match the return value type of the function (**int***). The function is syntactically correct.

```
int* foo(int i, int j)
{
  for(k=0; i < j; j++ ) if( i < j-
    2 )
        sum = sum+i
  return sum;
}
```

Role of the Parser

Not all sequences of tokens are program. Parser must distinguish between valid and invalid sequences of tokens. What we need is an expressive way to describe the syntax of programs and an acceptor mechanism that determines if input token stream satisfies the syntax of the programming language. The acceptor mechanism determines if input token stream satisfies the syntax of a programming language.

Parsing is the process of discovering a *derivation* for some sentence of a language. The mathematical model of syntax is represented by a grammar *G*. The language generated by the grammar is indicated by *L(G)*. Syntax of most programming languages can be represented by *Context Free Grammars* (CFG).

A CFG is a four tuple *G=(S,N,T,P)*

1. *S* is the *start symbol*
2. *N* is a set of *non-terminals*
3. *T* is a set of *terminals*
4. *P* is a set of *productions*

Why aren't Regular Expressions used to represent syntax? The reasons is that regular languages do not have enough power to express syntax of programming languages. Moreover, finite automaton can't remember number of times it has visited a particular state.

Consider the following example of CFG

SheepNoise ? SheepNoise baa |
baa

This CFG defines the set of noises sheep make. We can use the *SheepNoise* grammar to create sentences of the language. We use the productions as rewriting rules

Rule	*Sentential Form*
-	*SheepNoise*
1	*SheepNoise* *baa*
1	*SheepNoise* *baa baa*
2	*baa baa baa*

While it is cute, this example quickly runs out intellectual steam. To explore uses of CFGs, we need a more complex grammar. Consider the grammar for arithmetic expressions:

1 *expr*	**?**	*expr op expr*
2	**|**	num
3	**|**	id
4 *op*	**?**	+
5	**|**	–
6	**|**	*
7	**|**	/

Grammar rules in a similar form were first used in the description of the Algol-60 programming language. The syntax of C, C++ and Java is derived heavily from Algol-60.
The notation was developed by John Backus and adapted by Peter Naur for the Algol-60 language report; thus the term Backus-Naur Form (BNF)

Let us use the expression grammar to derive the sentence

x – 2 * y

Rule	*Sentential Form*
-	*expr*
1	*expr op expr*
2	<id,x> *op expr*
5	<id,x> – *expr*
1	<id,x> – *expr op expr*
2	<id,x> – <num,2> *op expr*
6	<id,x> – <num,2> ▯ *expr*
3	<id,x> – <num,2> ▯ <id,y>

Such a process of rewrites is called a *derivation* and the process or discovering a derivation is called *parsing.* At each step, we choose a non-terminal to replace. Different choices can lead to different derivations.
Two derivations are of interest

1. *Leftmost*: replace leftmost non-terminal (NT) at each step
2. *Rightmost*: replace rightmost NT at each step

The example on the preceding slides was *leftmost derivation.* There is also a *rightmost* derivation. In both cases we have

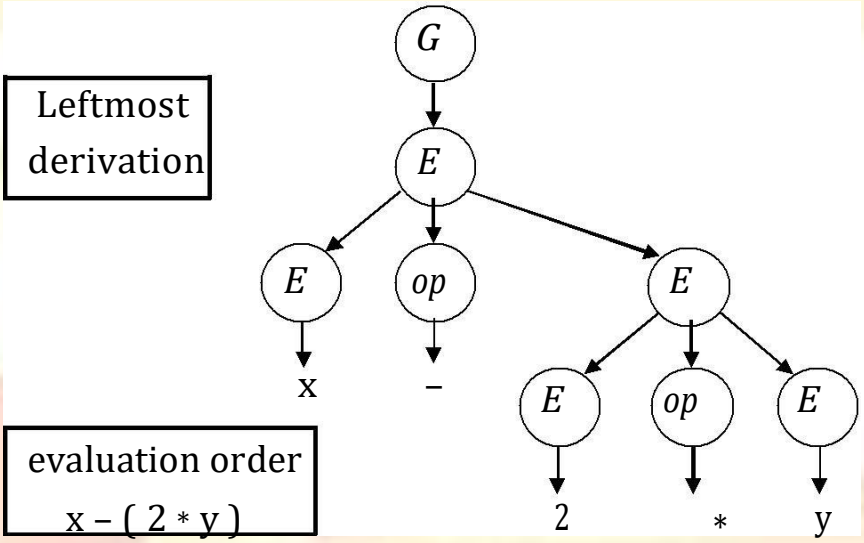
expr ? * id – num ▯ id

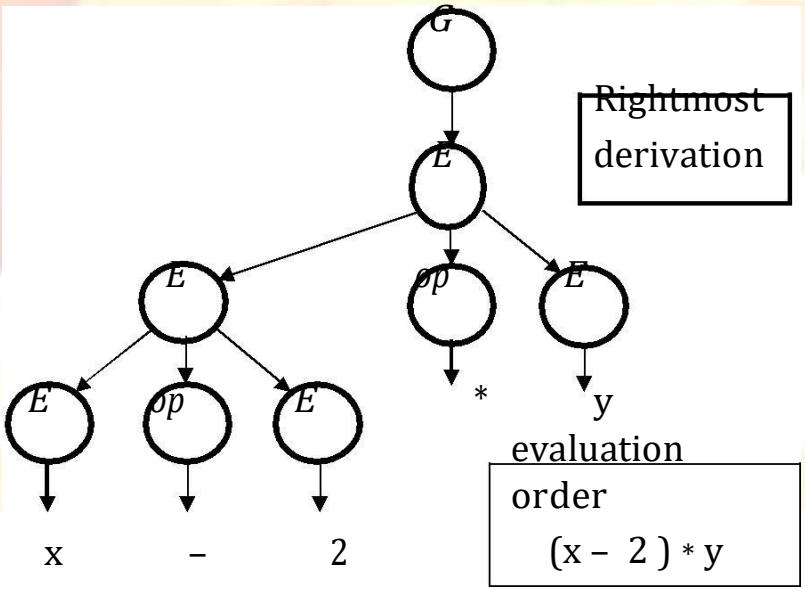
The two derivations produce different *parse trees*. The parse trees imply different evaluation orders!

Lecture 12

Parse Trees

The derivations can be represented in a tree-like fashion. The interior nodes contain the non-terminals used during the derivation





Precedence

These two derivations point out a problem with the grammar. It has no notion of *precedence*, or implied order of evaluation. The normal arithmetic rules say that multiplication has higher precedence than subtraction. To add precedence, create a non-terminal for each *level of precedence.* Isolate corresponding part of grammar to force parser to recognize high precedence sub-expressions first. Here is the revised grammar:

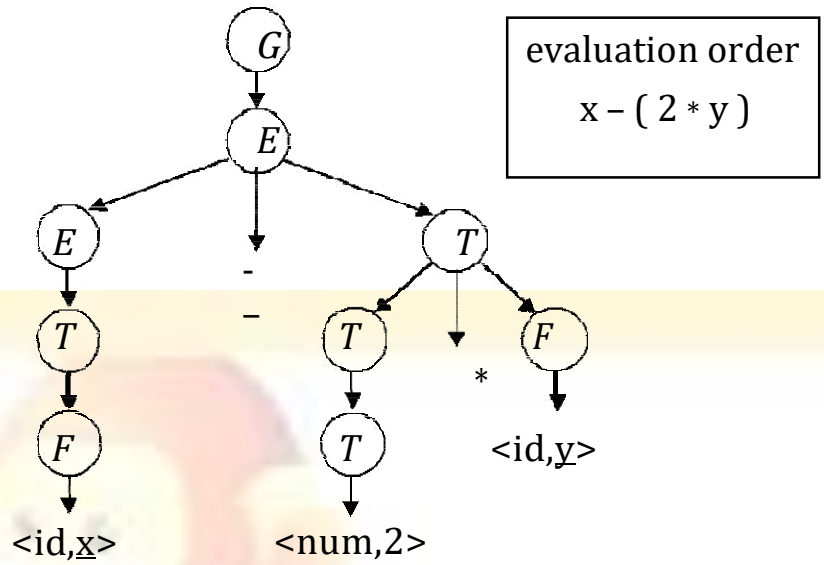
	1	*Goal*	?	*expr*
level two	2	*expr*	?	*expr* + *term*
	3		|	*expr* – *term*
	4		|	*term*
level one	5	*term*	?	*term* * *factor*
	6		|	*term* / *factor*
	7		|	*factor*
	8	*factor*	?	number
	9		|	Id

This grammar is larger and requires more rewriting to reach some of the terminal symbols. But it encodes expected precedence. Let's see how it parses

x – 2 * y

Rule	*Sentential Form*
-	*Goal*
1	*Expr*
3	*expr* – *term*
5	*expr* – *term* * *factor*
9	*expr* – *term* * <id,y>
7	*expr* – *factor* * <id,y>
8	*expr* – <num,2> * <id,y>
4	*term* – <num,2> * <id,y>
7	*factor* – <num,2> * <id,y>
9	<id,x> – <num,2> * <id,y>

This produces same parse tree under leftmost and rightmost derivations



Both leftmost and rightmost derivations give the *same expression* because the grammar directly encodes the desired precedence.

Ambiguous Grammars

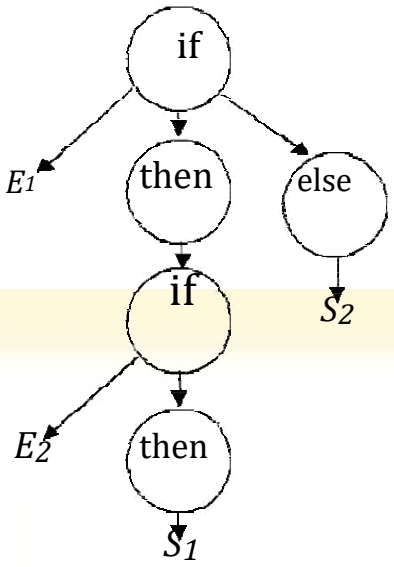
If a grammar has more than one leftmost derivation for a single sentential form, the grammar is *ambiguous.* The leftmost and rightmost derivations for a sentential form may differ, even in an *unambiguous* grammar. Let's consider the classic if-then-else example

Stmt → if *Expr* then *Stmt*
| if *Expr* then *Stmt* else *Stmt*
| … *other stmts* ….

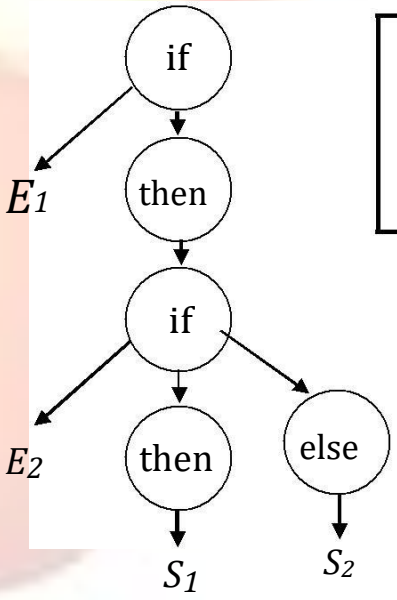
The following sentential form has two derivations:

if *E1* then if *E2* then *S* else *S2*

Production 1, then Production 2:
if E_1 then
  if E_2 then S_1
else S_2



Production 2, then Production 1:
if E_1 then
  if E_2 then S_1
  else S_2



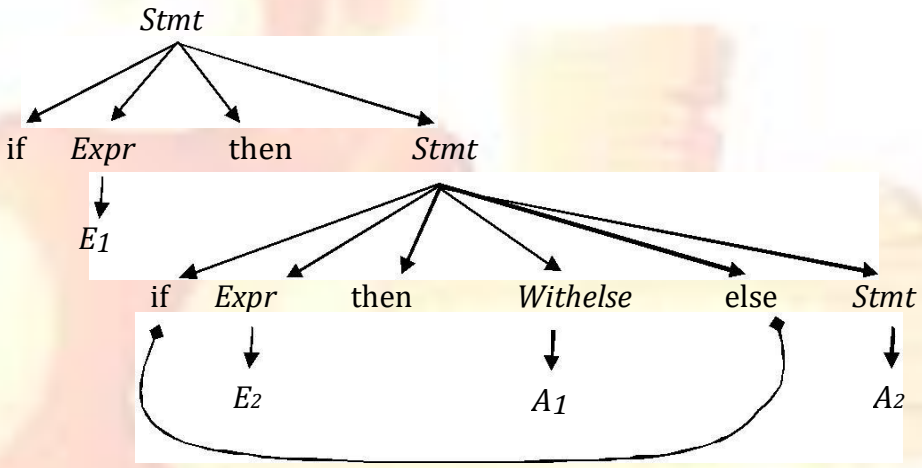
The convention in most programming languages is to match the else with the most recent if.

We can rewrite grammar to avoid generating the problem and match each else to innermost unmatched if:

1. Stmt	**?** If *E* then *Stmt*
2.	| If *E* then *WithElse* else *Stmt*
3.	| *Assignment*
4. WithElse	**?** If *E* then *WithElse* else *WithElse*
5.	| *Assignment*

Let derive the following using the rewritten grammar:

if E_1 then if E_2 then A_1 else A_2



This binds the else controlling A_2 to inner if

Context-Free Grammars

We have been using the term context- free without explaining why such rules are in fact "free of context". The simple reason is that non-terminals appear by themselves to the left of the arrow in context-free rules:

$A \rightarrow \alpha$

The rule $A \rightarrow \alpha$ says that *A* may be replaced by α *anywhere*, regardless of where *A* occurs. On the other hand, we could define a context as pair of strings β , γ, such that a rule would apply only if β occurs before and γ occurs after the non-terminal *A*. We would write this as

$\beta A \gamma \rightarrow \beta \alpha \gamma$

Such a rule in which α → ε is called a *context-sensitive* grammar rule. We would write this as $\beta A \gamma ? \beta \alpha \gamma$
Such a rule in which α → ε is called a *context-sensitive* grammar rule

Parsing Techniques

There are two primary parsing techniques: top-down and bottom- up.

Top-down parsers

A top-down parsers starts at the root of the parse tree and grows towards leaves. At each node, the parser picks a production and tries to match the input. However, the parser may pick the wrong production in which case it will need to backtrack. Some grammars are backtrack- free.

Bottom-up parsers

A bottom- up parser starts at the leaves and grows toward root of the parse tree. As input is consumed, the parser encodes possibilities in an internal state. The bottom- up parser starts in a state valid for legal first tokens. Bottom-up parsers handle a large class of grammars

Lecture 13

Top-Down Parser

A top-down parser starts with the root of the parse tree. The root node is labeled with the goal (start) symbol of the grammar. The top-down parsing algorithm proceeds as follows:

1. Construct the root node of the parse tree
2. Repeat until the fringe of the parse tree matches input string
    a. At a node labeled *A*, select a production with *A* on its lhs
    b. for each symbol on its rhs, construct the appropriate child
    c. When a terminal symbol is added to the fringe and it does not match the fringe, backtrack

The key is picking right production in step a. That choice should be guided by the input string. Let's try parsing using this algorithm using the expression grammar.

x – 2 * y

P	*Sentential Form*	*input*
-	*Goal*	x – 2 * y
1	*expr*	x – 2 * y
2	*expr + term*	x – 2 * y
4	*term + term*	x – 2 * y
7	*factor + term*	x – 2 * y
9	<id,x> *+ term*	x – 2 * y
9	<id,x> *+ term*	x □ – 2 * y

This worked well except that "–" does not match "+". The parser made the wrong choice of production to use at step 2. The parser must backtrack and use a different production.

P	*Sentential Form*	*input*
-	*Goal*	x – 2 * y
1	*expr*	x – 2 * y
3	*expr – term*	x – 2 * y
4	*term – term*	x – 2 * y
7	*factor – term*	x – 2 * y
9	<id,x> *– term*	x – 2 * y
9	<id,x> *– term*	x □ – 2 * y

This time the "–" and "–" matched. We can advance past "–" to look at "2". Now, we need to expand "*term*"

P	*Sentential Form*	*input*
-	<id,x> – *term*	x – 2 * y
7	<id,x> – *factor*	x – 2 * y
9	<id,x> – <num,2>	x – 2 * y
-	<id,x> – <num,2>	x – 2 * y

The 2's match but the expansion terminated too soon because there is still unconsumed input and there are no non-terminals to expand in the sentential form → Need to backtrack.

P	*Sentential Form*	*input*
-	<id,x> – *term*	x – 2 * y
5	<id,x> – *term* * *factor*	x – 2 * y
7	<id,x> – *factor* * *factor*	x – 2 * y
8	<id,x> – <num,2> * *factor*	x – 2 * y
-	<id,x> – <num,2> * *factor*	x – 2 * y
-	<id,x> – <num,2> * *factor*	x – 2 * y
9	<id,x> – <num,2> * <id,y>	x – 2 * y
-	<id,x> – <num,2> * <id,y>	x – 2 * y

This time the parser met with success. All of the input matched.

Left Recursion

Consider another possible parse:

P	*Sentential Form*	*input*
-	*Goal*	x – 2 * y
1	*expr*	x – 2 * y
2	*expr* + *term*	x – 2 * y
2	*expr* + *term* + *term*	x – 2 * y
2	*expr* + *term* + *term* + *term*	x – 2 * y
2	*expr* + *term* + *term* + *term* + ...	x – 2 * y

Parser is using productions but no input is being consumed.

Top-down parsers cannot handle left-recursive grammars. Formally, a grammar is left recursive if □ ∃ $A \in NT$ such that □ a derivation $A \Rightarrow^* A\alpha$, fc$\alpha \in (NT \cup T)^*$.

some string

Our expression grammar is left recursive. This can lead to non-termination in a top-down parser. Non-termination is bad in any part of a compiler! For a top-down parser, any recursion must be a *right recursion.* We would like to convert left recur sion to right

To remove left recursion, we can transform the grammar. Consider a grammar fragment:

$$A \;?\; A\alpha \mid \beta$$

Where neither α nor β starts with *A*. We can rewrite this as:

$$A \;?\; \beta A'$$

$$A' \;?\; \alpha A' \mid \varepsilon$$

where *A′* is a new non-terminal. This grammar accepts the same language but uses only right recursion. The expression grammar we have been using contains two cases of left-recursion. Applying the transformation yields

expr	?	*term expr′*
expr′	?	*+ term expr′*
	|	*– term expr′*
term	?	*factor term′*
term′	?	** factor term′*
		/ factor
	|	*term′*
	|	*ε*

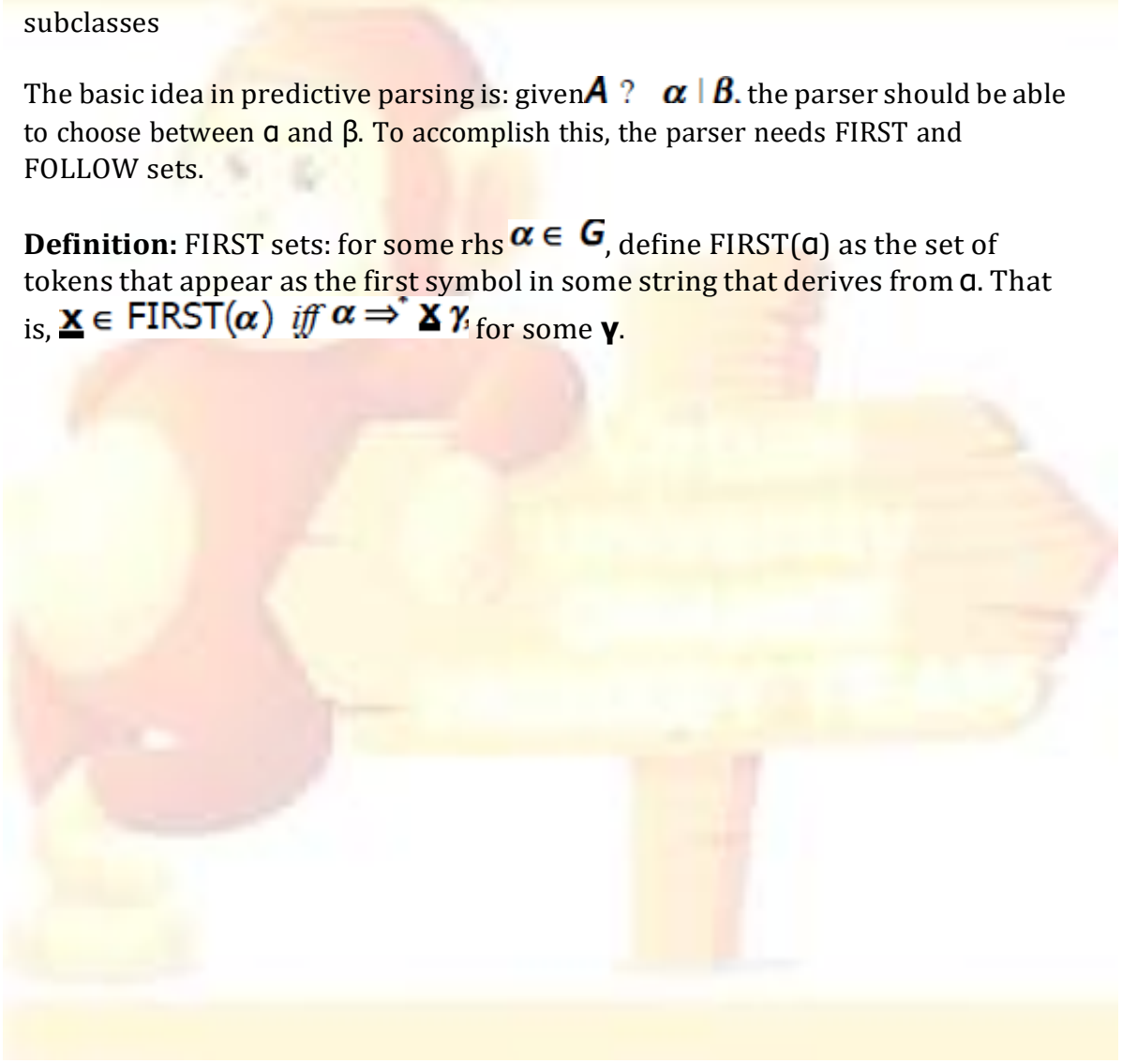
These fragments use only right recursion. They retain the original left associativity. A top-down parser will terminate using them.

Predictive Parsing

If a top down parser picks the wrong production, it may need to backtrack. Alternative is to *look-ahead* in input and use context to pick the production to use correctly. How much look-ahead is needed? In general, an arbitrarily large amount of
look-ahead symbols are required.. Fortunately, large classes of CFGs can be parsed with limited lookahead. Most programming languages constructs fall in those subclasses

The basic idea in predictive parsing is: given $A \rightarrow \alpha \mid \beta$. the parser should be able to choose between α and β. To accomplish this, the parser needs FIRST and FOLLOW sets.

Definition: FIRST sets: for some rhs $\alpha \in G$, define FIRST(α) as the set of tokens that appear as the first symbol in some string that derives from α. That is, $x \in \text{FIRST}(\alpha)$ *iff* $\alpha \Rightarrow^* x\gamma$, for some **γ**.



Lecture 14

The LL(1) Property

If A ? α and A ? β both appear in the grammar, we would like

$$FIRST(\alpha) \cap FIRST(\beta) = \varnothing$$

Predictive parsers accept LL(k) grammars. The two LL stand for Left-to-right scan of input, left- most derivation. The k stands for number of look-ahead tokens of input. The LL(1) Property allows the parser to make a correct choice with a look-ahead of exactly *one symbol*! What about < - productions? They complicate the definition of LL(1).
If A ? α and A ?β and ε ∈ FIRST(α) , then we need to ensure that FIRST(β) is disjoint from FOLLOW(α), too.

Definition: FOLLOW(α) is the set of all words in the grammar that can legally appear after an α.

For a non-terminal *X*, FOLLOW(*X*) is the set of symbols that might follow the derivation of *X*. Define $FIRST^+(\alpha)$ as FIRST(α) ∪ FOLLOW(α), if ε ∪ FIRST(α), FIRST(α), otherwise. Then a grammar is LL(1) *iff* A ? α and A ? β implies

$$FIRST^+(\alpha) \cap FIRST^+(\beta) = \varnothing$$

Given a grammar that has the is LL(1) property, we can write a simple routine to recognize each lhs. The code is simple and fast.
Consider

A ? $\beta_1 | \beta_2 | \beta_3$,

which satisfies the LL(1) property $FIRST^+(\alpha) \cap FIRST^+(\beta) = \varnothing$

```
/* find an A */
if(token ∈ FIRST(β1))
  find a β1 and return true
else if(token ∈ FIRST(β2))
  find a β2 and return true
if(token ∈ FIRST(β3))
  find a β3 and return true
else error and return false
```

Grammar with the LL(1) property are called *predictive grammars* because the parser can "predict" the correct expansion at each point in the parse. Parsers that capitalize on the LL(1) property are called *predictive parsers.* One kind of predictive parser is the *recursive descent* parser.

Recursive Descent Parsing

Consider the right-recursive expression grammar

1	*Goal*	?	*expr*
2	*expr*	?	*term expr'*
3	*expr'*	?	*+ term expr'*
4		|	*- term expr'*
5		|	***ε***
6	*term*	?	*factor term'*
7	*term'*	?	** factor term'*
8		|	*/ factor term'*
9		|	***ε***
10	*factor*	?	number
11		|	id
12		|	(*expr*)

This leads to a parser with six mutually recursive routines: goal, expr, eprime, term, tprime and factor. Each recognizes one non-terminal (NT) or terminal (T). The term *descent* refers to the direction in which the *parse tree* is built. Here are some of these routines written as functions:

```
Goal() {
    token = next_token();
    if(Expr() == true && token == EOF)
        next compilation step
    else {
        report syntax error;
        return false;
    }
}
Expr()
{
    if(Term() == false)
        return false;
    else
        return Eprime();
}
Eprime() {
    token_type op = next_token(); if( op ==
    PLUS || op == MINUS ) {
        if(Term() == false)
            return false;
```

```
            else
                return Eprime();
        }
    }
```

Functions for other non-terminals Term, Factor and Tprime follow the same pattern.

Recursive Descent in C++

This form of the routines is too procedural. Moreover, there is no convenient way to build the parse tree. We can use C++ to code the recursive descent parser in an object oriented manner. We associate a C++ class with each non-terminal symbol. An instantiated object of a non-terminal class contains pointer to the parse tree. Here is the C++ code for the non-terminal classes:

```
class NonTerminal {
public:
        NonTerminal(Scanner* sc)
        {
            s = sc; tree = NULL;
        }
        virtual ~NonTerminal(){}
        virtual bool isPresent()=0;
        TreeNode* AST(){
            return tree;
        }

protected:
        Scanner
        * s;
        TreeNode* tree;
}

class Expr:public NonTerminal
{
public:
    Expr(Scanner* sc): NonTerminal(sc){ } virtual
    bool isPresent();
}

class Eprime:public NonTerminal {
public:
  Eprime(Scanner* sc, TreeNode* t): NonTerminal(sc)
```

```
  {
            exprSofar = t;
  }
  virtual bool isPresent();
protected:
  TreeNode* exprSofar;
}

class Term:public NonTerminal
{
public:
    Term(Scanner* sc): NonTerminal(sc){ } virtual bool
    isPresent();
}

class Tprime:public NonTerminal { public:
    Tprime(Scanner* sc, TreeNode* t): NonTerminal(sc)
    {
        exprSofar = t;
    }
    virtual bool isPresent(); protected:
    TreeNode* exprSofar;
}

class Factor:public NonTerminal { public:
    Factor(Scanner* sc, TreeNode* t): NonTerminal(sc){ };

    virtual bool isPresent();
}
```

Lecture 15

Let's consider the implementation of the C++ classes for the non-terminals. We start with

Expr.

```
bool Expr::isPresent()
{
      Term* op1 = new Term(s);
      if(!op1->isPresent())
            return false;
      tree = op1->AST();
      Eprime* op2 = new Eprime(s, tree);
      if(op2->isPresent())
          tree = op2->AST();
      return true;
}

bool Eprime::isPresent()
{
 int op=s->nextToken();
 if(op==PLUS || op==MINUS){
  s->advance();
  Term* op2=new Term(s);
  if(!op2->isPresent())
     syntaxError(s);
  TreeNode* t2=op2->AST();
  tree = new TreeNode(op,exprSofar,t2);
  Eprime* op3 = new Eprime(s, tree);
  if(op3->isPresent())
     tree = op3->AST();
     return true;
  }
 else return false;
}

bool Term::isPresent()
{
 Factor* op1 = new Factor(s);
 if(!op1->isPresent())
     return false;
 tree = op1->AST();
 Tprime* op2 = new Tprime(s, tree);
 if(op2->isPresent())
     tree = op2->AST();
 return true;
}
```

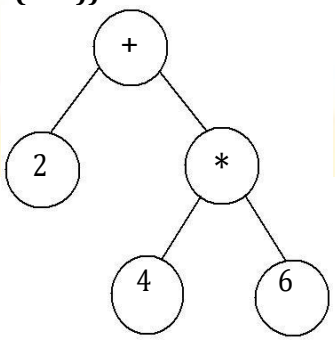
```
bool Tprime::isPresent()
{
 int  op=s->nextToken();
 if(op == MUL || op == DIV){
  s->advance();
  Factor* op2=new Factor(s);
  if(!op2->isPresent())
     syntaxError(s);
  TreeNode* t2=op2->AST();
  tree = new TreeNode(op,exprSofar,t2);
  Tprime* op3 = new Tprime(s, tree);
  if(op3->isPresent())
     tree = op3->AST();
     return true;
  }
 else return false;
}
bool Factor::isPresent()
{
 int op=s->nextToken();
 if(op == ID || op == NUM)
 {
  tree = new TreeNode(op,s->tokenValue());
  s->advance();
  return true;
 }
 if( op == LPAREN ){
  s->advance();
  Expr* opr = new Expr(s);
  if(!opr->isPresent() )
   syntaxError(s);
  if(s->nextToken() != RPAREN)
   syntaxError(s);
  s->advance();
  tree = opr->AST();
  return true;
 }
 return false;
}
```

Lecture 16

Here is the output trace for the expression: **2+4*6**

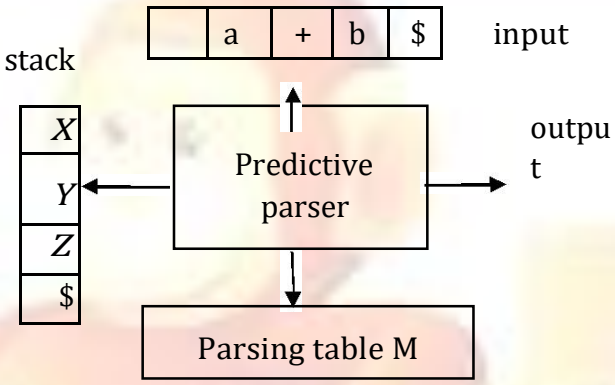
```
>> Expr::isPresent()
   >> Term::isPresent()
      >> Factor::isPresent() token: 2
                    (257)
      <<  Factor::isPresent() return true
      >> Tprime::isPresent()
                    token: + (267)
      << Tprime::isPresent() return false
   <<  Term::isPresent() return true
   >> Eprime::isPresent()
            token: + (267)
      >>  Term::isPresent()
         >> Factor::isPresent() token: 4
                    (257)
         << Factor::isPresent() return true >>
         Tprime::isPresent()
                    token: * (269)
            >> Factor::isPresent() token: 6
                    (257)
            <<  Factor::isPresent() return true
            >> Tprime::isPresent()
                    token:      (0)
            <<  Tprime::isPresent() return false
         <<  Tprime::isPresent() return true
      <<  Term::isPresent() return true
      >> Eprime::isPresent()
                 token:      (0)
      << Eprime::isPresent() return false
   <<  Eprime::isPresent() return true
<<  Expr::isPresent() return true

**   AST ** (2+(4*6))
```



Non-recursive Predictive Parsing

It is possible to build a non-recursive predictive parser. This is done by maintaining an explicit stack and using a table. Such a parser is called a table-driven parser. The non-recursive LL(1) parser looks up the production to apply by looking up a parsing table. The LL(1) table has one dimension for current non-terminal to expand and another dimension for next token. Each table cell contains one production.



Consider the expression grammar

1	E	**?**	$T\ E'$
2	E'	**?**	$+\ T\ E'$
3		|	ε
4	T	**?**	$F\ T'$
5	T'	**?**	$*\ F\ T'$
6		|	ε
7	F	**?**	$(\ E\ )$
8		|	id

Using the table construction algorithm that will be discussed later, we have the predictive parsing table

	id	+	*	(	)	$
E	E **?** TE'			E **?** TE'		
E'		E' **?** $+TE'$			E' **?** ε	E' **?** ε
T	T **?** FT'			T **?** FT'		
T'		T' **?** ε	T **?** $*FT'$		T' **?** ε	T' **?** ε
F	F **?** id			F **?** (E)		

The rows are non-terminals and the columns are the terminals of the expression grammar. The predictive parser uses an explicit stack to keep track of pending non-terminals. It can thus be implemented without recursion.

LL(1) Parsing Algorithm

The *input buffer* contains the string to be parsed; $ is the end-of- input marker. The *stack* contains a sequence of grammar symbols. Initially, the stack contains the start symbol of the grammar on the top of $. The parser is controlled by a program that behaves as follows:

The program considers *X*, the symbol on top of the stack, and a, the current input symbol. These two symbols, *X* and a determine the action of the parser. There are *three* possibilities.

1. *X* = a = $, the parser halts and announces successful completion.
2. *X* = a ≠ $ the parser pops *X* off the stack and advances input pointer to next input symbol.
3. If *X* is a non-terminal, the program consults entry M[*X*,a] of parsing table M.
    a. If the entry is a production M[*X*,a] = {*X* ? *UVW* }, the parser replaces *X* on top of the stack by *WVU* (with *U* on top). As output, the parser just prints the production used: *X* ? *UVW.* However, any other code could be executed here.
    b. If M[*X*,a] =**error**, the parser calls an error recovery routine

Example: let's parse the input string

id+idid

using the non-recursive LL(1) parser

Stack	*Input*	*Ouput*
$*E*	id+id id$	
$*E′ T*	id+id id$	*E* **?** *TE′*
$*E′ T′ F*	id+id id$	*T* **?** *FT′*
$*E′T* ′ id	id+id id$	*F* **?** id
$*E′* *T′*	+id id$	
$*E′*	+id id$	*T′* **?** **ε**
$*E′ T* +	+id id$	*E′* **?** +*TE′*
$*E′ T*	id id$	
$*E′ T′ F*	id id$	*T* **?** *FT′*
$*E′* *T′* id	id id$	*F* **?** id
$*E′* *T′*	id$	
$*E′ T′ F*	id$	*T* **?** *FT′*
$*E′ T′ F*	id$	
$*E′T* ′ id	id$	*F* **?** id
$*E′* *T′*	$	
$*E′*	$	*T′* **?** **ε**
$	$	*E′* **?** **ε**

Lecture 17

Note that productions output are tracing out a *leftmost derivation*. The grammar symbols on the stack make up *left-sentential forms.*

LL(1) Table Construction

Top-down parsing expands a parse tree from the start symbol to the leaves. It always expands the leftmost non-terminal. Consider the state

$S ? ^* \beta A \gamma$

with b the next token and we are trying to match $\beta b \gamma$. There are two possibilities

1. b belongs to an expansion of A.
   Any A ? α can be used if b can start a string derived from α In this case we say that $b \in FIRST(\alpha)$
2. b does not belong to an expansion of A. Expansion of A is empty, i.e., A ? **ε** and b belongs an expansion of γ, e.g., $b\omega$. which means that b can appear after A in a derivation of the form $S ? ^* \beta A b \omega$. We say that $b \in FOLLOW(A)$.

Any A ? **α** can be used if **α** expands to **ε** We say that $\varepsilon \in FIRST(A)$ in this case.

Definition

$$FIRST(X) = \{ b \mid X ? ^* ba \} \cup \{ \varepsilon \mid X ? ^* \varepsilon \}$$

Lecture 18

Computing FIRST Sets

Here is the algorithm for computing the FIRST sets.

1. For all terminal symbols b,

   FIRST(b) = {b}

2. For all productions: $X \to A_1 \ldots A_n$

   Add FIRST(A_1) – {ε} to FIRST(X), stop if ε ∉ FIRST(A_1)
   Add FIRST(A_2) – {ε} to FIRST(X), stop if ε ∉ FIRST(A_2)
   .....
   Add FIRST(A_n) – {ε} to FIRST(X), stop if ε ∉ FIRST(A_n)
   Add ε to FIRST(X)

This strategy is encoded in the following procedure

```
for each a ∈ ( T ∪ ε )
        FIRST(a) ? {a}
for each A ∈ NT
        FIRST(A) ? Ø
while ( FIRST sets are still changing)
        for each A ? β1β2... βk ∈ P
                FIRST(A) ? FIRST(A) ∪ (FIRST(β1) – {ε})
                i ? 1
                while ( ε ∈ FIRST(βi) and i = k-1 )
                        FIRST(A) ? FIRST(A) ∪ (FIRST(βi+1) – {ε})
                        i ? i+1
                if (i == k and ε ∈ FIRST(βk)
                        FIRST(A) ? FIRST(A) ∪ {ε}
```

Example : consider the expression grammar again

1	E	?	$T\ E'$
2	E'	?	$+\ T\ E'$
3		|	ε
4	T	?	$F\ T'$
5	T'	?	$*\ F\ T'$
6		|	ε
7	F	?	$(\ E\ )$
8		|	Id

FIRST(id) = { id }
FIRST('(') = { (}
FIRST('+') = { + }

FIRST(*E*) = {FIRST(*T*) – {ε} }
FIRST(*T*) = {FIRST(*F*) – {ε} }
FIRST(*F*) = {FIRST('(') – {ε} } = { (}

FIRST(*F*) = { '(' } + {FIRST(id) – {ε} } = { (, id}

FIRST(*E'*) = { +, ε }
FIRST(*T'*) = { *, ε }

Thus,

FIRST(*E*) = FIRST(*T*) = FIRST(*F*) = { (, id }
FIRST(*E'*) = { +, ε }
FIRST(*T'*) = { *, ε }

FOLLOW Sets

Definition:

FOLLOW(X) = { b | S ? * βXbω}

Computing FOLLOW Sets:

1. Add $ to FOLLOW(S) where S is the start non-terminal.
2. If there is a production A ? αBβ, then everything in FIRST(β) – {ε} is in FOLLOW(B).
3. If there is a production A ? αB, or A ? αBβ, where in ε ∈ FIRST(β) (i.e., β ? * ε), then everything in FOLLOW(A) is in FOLLOW(B)

The following procedure encodes this strategy.

```
for each A ∈ NT
        FOLLOW(A) ? Ø
FOLLOW(S) ? {$}
while ( FOLLOW sets are still changing)
    for each A ? β1β2... βk ∈ P
        FOLLOW(βk) ? FOLLOW(βk) ∪ FOLLOW(A)
        T ? FOLLOW(A)
        for i ? k downto 2
         if ( ε ∈ FIRST(βi) )
            FOLLOW(βi-1) ? FOLLOW(βi-1) ∪ (FIRST(βi) – {ε}) ∪ T
         else
            FOLLOW(βi-1) ? FOLLOW(βi-1) ∪FIRST(βi)
            T ? Ø
```

Let's apply the algorithm to the expression grammar.

Put $ in FOLLOW(E). By rule (2) applied to production **?** (E), ')' is also in FOLLOW(E). Thus, FOLLOW(E) = {), $}. By rule (3) applied to production
E **?** T E' , $ and ')' are in FOLLOW(E'). Thus, FOLLOW(E) = FOLLOW (E') = {), $ }. Similarly, FOLLOW(T) = FOLLOW (T') = { +,), $ } and FOLLOW(F) = { +, |,), $ }

Lecture 19

LL(1) Table Construction

Here now is the algorithm to construct a predictive parsing table.

1. For each production $A \rightarrow \alpha$

   1. for each terminal a in FIRST(α), add $A \rightarrow \alpha$ to M[A,a].

   2. If ε is in FIRST(α), add $A \rightarrow \alpha$ to M[A,b] for each terminal b in FOLLOW(A). If ε is in FIRST(α), and $ is in FOLLOW(A), add $A \rightarrow \alpha$ to M[A,$].

2. Make each undefined entry of M be error.

Let us apply the algorithm to the expression grammar. Since FIRST(TE') = FIRST(T) = { (, id }, the production $E \rightarrow TE'$ cause M[E,(] and M[E,id] to get $E \rightarrow TE'$. The production $E' \rightarrow +TE'$ causes M[E',+] to get $E' \rightarrow +TE'$. The production $E' \rightarrow \varepsilon$ causes M[E',)] and M[E',$] to get $E' \rightarrow \varepsilon$ since FOLLOW (E') = {), $ }. And so on. The final parsing table produced is:

	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow$ id			$F \rightarrow (E)$		

Left Factoring

Consider the grammar

E → T + E | T
T → int | int T | (E)

It is impossible to predict because for T, two productions start with int. For E, it

is not clear how to predict; the two productions start with the non-terminal T. A grammar must be *left factored* before use for predictive parsing. The procedure to left-factor a grammar is as follows:

If $\alpha \neq \varepsilon$, replace all productions

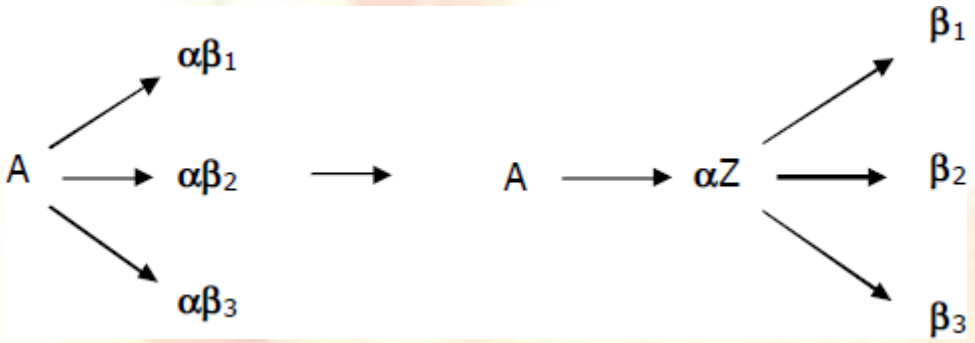
$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma$$

with

$$A \rightarrow \alpha Z \mid \gamma$$
$$Z \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$$

where Z is a new non-terminal

A graphical explanation:



Example: consider following fragment of expression grammar

```
Factor ?    id
        |   id [ ExprList ]
        |   id ( ExprList )
```

After left factoring, the grammar becomes

```
Factor  ? id Args
Args    ? [ ExprList ]
        |  ( ExprList )
        |  ε
```

Given a CFG that does not meet the LL(1) condition, it is *un-decidable* whether or not an equivalent LL(1) grammar exists.

Lecture 20

Bottom-up Parsing

Bottom-up parsing is more general than top-down parsing. Bottom-up parsers handle a large class of grammars. It is the preferred method in practice. It is also called *LR* parsing; *L* means that tokens are read left to right and *R* means that the parser constructs a rightmost derivation. LR parsers do not need left-factored grammars. LR parsers can handle left-recursive grammars.

LR parsing *reduces* a string to the start symbol by inverting productions. A derivation consists of a series of rewrite steps

$$S \Rightarrow \gamma_0 \Rightarrow \gamma_1 \Rightarrow \ldots \Rightarrow \gamma_{n-1} \Rightarrow \gamma_n \Rightarrow \text{sentence}$$

Each γ_i is a *sentential form.* If γ contains only terminals, γ is a *sentence* in L(G). If γ contains ≥ 1 nonterminals, γ is a *sentential form*. A bottom-up parser builds a derivation by working from input sentence *back* towards the start symbol S.

Consider the grammar

S → aABe
A → Abc | b
B → d

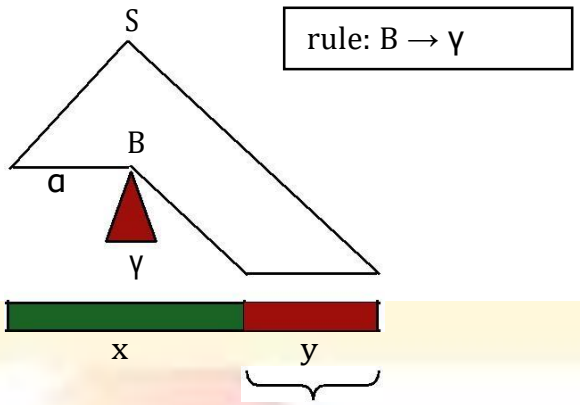
The sentence abbcde can be reduced to S:

abbcde
aAbcd
e aAde
aABe S

These reductions, in fact, trace out the following right-most derivation in reverse:

S → aABe
→aAde
→aAbcde
→abbcde

S →aBy →aγy →xy



Terminals only

Consider the grammar

1. E → E + (E)
2. | int

The bottom-up parse of the string int + (int) + (int) would be

int + (int) + (int)
E + (int) + (int) E
+ (E) + (int)
E + (int)
E + (E)
E

The consequence of an LR parser tracing a rightmost derivation in reverse is that given αβγ be a step of a bottom-up parse, assuming that next reduction is A → β Then γ is a string of terminals. The reason is that αAγ → αβγ is a step in a rightmost derivation. This observation provides a strategy for building bottom up parsers: split the input string into two substrings. Right substring (a string of terminals) is as yet unexamined by parser and left substring has terminals and non-terminals. The dividing point is marked by a ▶(the ▶is not part of the string). Initially, all input is unexamined: ▶x_1 x_1 . . . x_n.

Shift-Reduce Parsing

Bottom-up parsing uses only two kinds of actions:

1. Shift
2. Reduce

Shift moves ▶one place to the right which shifts a terminal to the left stringE

+ (▶int) ⇒ E + (int ▶)

In the *reduce* action, the parser applies an inverse production at the right end of the left string. If E → E + (E) is a production, then

E + (E+(E)▶) ⇒ E + (E ▶)

Shift-Reduce Example

▶ int + (int) + (int) $	shift
int ▶+ (int) + (int) $	reduce E → intE
▶+ (int) + (int) $	shift 3 times
E + (int ▶) + (int) $	reduce E → int
E + (E ▶) + (int) $	shift
E + (E) ▶+ (int) $	reduce E → E+(E)
E ▶+ (int) $	shift 3 times
E + (int ▶) $	reduce E → int
E + (E ▶) $	shift
E + (E) ▶$	red E → E+(E)
E ▶$	*accept*

Lecture 21

Shift-Reduce: The Stack

A stack can be used to hold the content of the left string. The Top of the stack is marked by the ►symbol. The shift action pushes a terminal on the stack. Reduce pops zero or more symbols from the stack (production *rhs*) and pushes a non-terminal on the stack (production *lhs*)

Discovering Handles

A bottom-up parser builds the parse tree starting with its leaves and working toward its root. The upper edge of this partially constructed parse tree is called its *upper frontier*. At each step, the parser looks for a section of the upper frontier that matches right-hand side of some production. When it finds a match, the parser builds a new tree node with the production's left-hand non-terminal thus extending the frontier upwards towards the root. The critical step is developing an efficient mechanism that finds matches along the tree's current frontier.

Formally, the parser must find some substring β, of the upper frontier where

1. β is the right-hand side of some production $A \rightarrow \beta$, and
2. $A \rightarrow \beta$ is one step in right-most derivation of input stream

We can represent each potential match as a pair $\langle A \rightarrow \beta, k \rangle$, where k is the position on the tree's current frontier of the right-end of β. The pair $\langle A \rightarrow \beta, k \rangle$ is called the *handle* of the bottom-up parse.

Handle Pruning

A bottom-up parser operates by repeatedly locating handles on the frontier of the partial parse tree and performing reductions that they specify. The bottom-up parser uses a stack to hold the frontier. The stack simplifies the parsing algorithm in two ways.

First, the stack trivializes the problem of managing space for the frontier. To extend the frontier, the parser simply pushes the current input token onto the top of the stack. Second, the stack ensures that all handles occur with their right end at the top of the stack. This eliminates the need to represent handle's position.

Shift-Reduce Parsing Algorithm

```
push $ onto stack
sym ←
nextToken()
repeat until (sym == $ and the stack contains exactly Goal on top
        of $) if a handle for A → β on top of stack
                pop |β| symbols off the
                stack push A onto the
                stack
        else if (sym ≠ $ )
         push sym onto stack sym
         ← nextToken() else /* no
                handle, no input */
                report error and halt
```



Lecture 22

Example: here is the bottom-up parser's action when it parses the expression grammar sentence

x – 2 × y

(tokenized as id – num * id)

	word	Stack	Handle	Action
1	id	▶	- none -	*shift*
2	–	id ▶	<Factor → id,**1**>	*reduce*
3	–	Factor ▶	<Term → Factor,**1**>	*reduce*
4	–	Term ▶	<Expr → Term,**1**>	*reduce*
5	–	Expr ▶	- none -	*shift*
6	num	Expr – ▶	- none -	*shift*
7	×	Expr – num ▶	<Factor → num,**3**>	*reduce*
8	×	Expr – Factor ▶	<Term → Factor,**3**>	*shift*
9	×	Expr – Term ▶	- none -	*shift*
10	id	Expr – Term × ▶	- none -	*shift*
11	$	Expr – Term × id▶	<Factor → id,**5**>	*reduce*
12	$	Expr–Term × Factor▶	<Term→Term×Factor,**5**>	*reduce*
13	$	Expr – Term ▶	<Expr → Expr – Term,**3**>	*reduce*
14	$	Expr ▶	<Goal → Expr,**1**>	*reduce*
15	$	Goal	- none -	***accept***

Handles

The handle-finding mechanism is the key to efficient bottom-up parsing. As it process an input string, the parser must find and track all potential handles. For example, every legal input eventually reduces the entire frontier to grammar's goal symbol. Thus,

<Goal → Expr,1> is a potential handle at the start of every parse. As the parser builds a derivation, it discovers other handles. At each step, the set of potential handles represent different suffixes that lead to a reduction. Each potential handle represent a string of grammar symbols that, if seen, would complete the right-hand side of some production.

For the bottom-up parse of the expression grammar string, we can represent the potential handles that the shift-reduce parser should track. Using the placeholder • to represent top of the stack, there are nine handles:

	Handles
1	<Factor → id •>
2	<Term → Factor•>
3	<Expr → Term •>
4	<Factor → num•>
5	<Term → Factor•>
6	<Factor → id •>
7	<Term → Term × Factor •>
8	<Expr → Expr – Term •>
9	<Goal → Expr •>

This notation shows that the second and fifth handles are identical, as are first and sixth. It also create a way to represent the potential of discovering a handle in future. Consider the parser's state in step 6: The parser has recognized Expr –. Using the stack-relative notation, we can represent the parser's state as Expr → Expr – • Term. The parser has already recognized an Expr and a –. If the parser reaches a state where it shifts a Term on top of Expr and –, it will complete the handle Expr → Expr – Term •. How many potential handles must the parser recognize? The right-hand side of each production can have a placeholder at its start, at its end and between any two consecutive symbols.

Expr → • Expr –Term
Expr → Expr • – Term
Expr → Expr – •Term
Expr → Expr – Term •

If the right-hand side of a production has *k* symbols, it has *k +1* placeholder positions.

Lecture 23

Handles

The number of potential handles for the grammar is simply the sum of the lengths of the right-hand side of all the productions. The number of complete handles is simply the number of productions. These two facts lead to the critical insight behind LR parsers:

> *A given grammar generates a finite set of handles (and potential handles) that the parser must recognize*

However, it is not a simple matter of putting the placeholder in right-hand side to generate handles. The parser needs to recognize the correct handle by different right contexts. Consider the parser's action at step 9.

	word	**Stack**	**Handle**	**Action**
9	×	Expr – Term ▶	- none -	*shift*
10	id	Expr – Term × ▶	- none -	*shift*
11	$	Expr – Term × id▶	<Factor → id,**5**>	*reduce*
12	$	Expr–Term × Factor▶	<Term→Term×Factor,**5**>	*reduce*
13	$	Expr – Term ▶	<Expr → Expr – Term,**3**>	*reduce*
14	$	Expr ▶	<Goal → Expr,**1**>	*reduce*
15	$	Goal	- none -	***accept***

The frontier is Expr – Term, suggesting a handle <Expr → Expr – Term>•. However, the parser decides to extend the frontier by shifting × on to the stack rather than reducing frontier to Expr. Clearly, this the correct move for the parser. No potential handle contains Expr followed by ×. At step 9, the set of potential handles is

<Expr → Expr – Term•>
<Term →Term• × Factor>
<Term →Term• / Factor >

The next input symbol clearly matches the second choice. The parser needs a basis for deciding between first (reduce) and second (shift) choices:

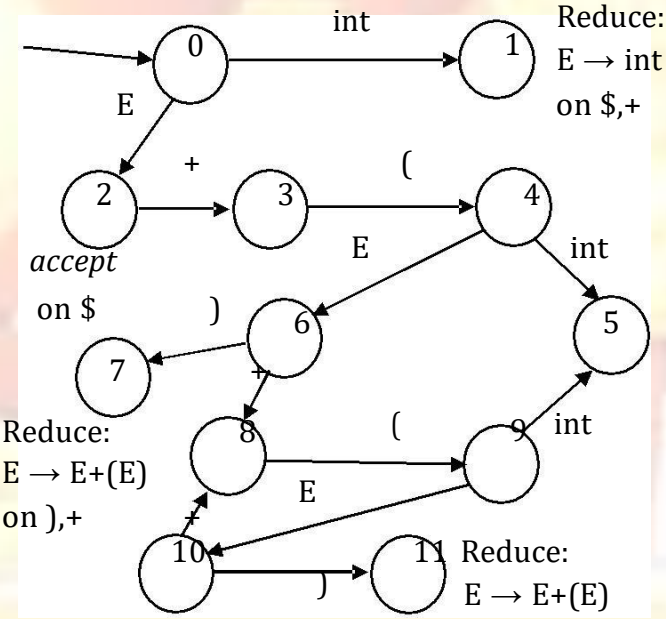
<Expr → Expr – Term•>
<Term →Term• × Factor>

This requires more context than the parser has in the frontier (stack). To choose between reducing and shifting, the parser must recognize which symbols can occur to the right of Expr and Term in valid phrases.

LR(1) Parsers

The LR(1) parsers can recognize precisely those languages in which one-symbol lookahead suffices to determine whether to shift or reduce. The LR(1) construction algorithm builds a handle-recognizing DFA. The parsing algorithm uses this DFA to recognize handles and potential handles on the parse stack

Parsing DFA



In order to remember the state the DFA goes into on a symbol, the parser stores the DFA state in the stack along with the symbol. Initial entry in the stack will be '<dummy,0>'.

Parsers represent DFA as a 2D table. The rows correspond to DFA states and columns correspond to terminals and non-terminals. The columns with

terminals and the rows form the *action table* while the columns with non-terminals and rows are called the *goto table*. It is customary to show these tables together.

Building LR(1) Tables

To construct *Action* and *Goto* tables, the LR(1) parser generator builds a model of handle-recognizing DFA. The model is used to fill in the tables. The LR(1)-table construction needs a concrete representation for the handles and their associated lookahead symbols. We call this representation an *LR(1) item*.



Lecture 24

An LR(1) item is a pair [X → α•β, a] where X → αβ is a production and a ∈ T (terminals) is look-ahead symbol. The model uses a set of LR(1) items to represent each parser state. The model is called the *canonical collection (CC) of set of LR(1) items.*

Canonical Collection

Each set in *CC* represents a state in the eventual parser DFA. The construction of CC begins by building a model of parser's initial state. The initial state consists of the set of LR(1) items that represent the parser's initial state, along with any items that must also hold in the initial state. To simplify the task of building this initial state, the construction requires that the grammar have a unique goal symbol. The convention is to add a new start symbol S to grammar and a production

S → E

This leads to the augmented grammar

S → E
E → E + (E) | int

The Closure Procedure

The item [S → •E, $] describes the parser's initial state. It represents a configuration in which recognizing S followed by $ would be a valid parse. This item, i.e., [S → •E, $] becomes the core of the first state in CC, labeled I_0. If the grammar has several distinct productions for the start symbol, each of them generates an item in this initial core of I_0. The procedure *closure* does this.

closure(*s*)
= *repeat*
 for each [X → α•Yβ, a] ∈ *s*
 for each *production* Y → α
 for each b ∈ FIRST(βa)
 s ← *s* ∪ [Y → • γ,
 b] *until s is unchanged*

Let's apply this procedure to the augmented grammar.

The first set is I_0 = *closure*({[S → •E, $] }) . Equating the terms in the

procedure, *s* = {[S → •E, $]}, [X → α •Yβ, **a**] ⇔ [S → •E, $], X = S, α = ε, Y = E,
β = ε, a = **$, Y** → γ ∪ E → E + (E) and E → intFIRST(βa) = FIRST($) = $.

This leads to expansion of *s*.

s = { [S → •E, $] } ∪ { [E → •E + (E), $] } ∪ { [E → •int, $] } =
{ [S → •E, $] , [E → •E + (E), $] , [E → •int, $] }

The set *s* changed so we repeat. The item [S → •E, $] is already processed. The for loop considers [X → α•Yβ, a] ⇔ [E→•E+(E),$], which leads to the match up X = E,
α= ε, Y = E, β= +(E), a = $, **Y** → γ ⇔ E → E + (E), ⇔ E → int
FIRST(βa) = FIRST(+(E)$) = +. The set *s* is extended

s = *s* ∪ { [E → •E+(E), +] } ∪ { [E → •int, +] }